Guía de Estudio Práctica — Claude Certified Architect (Foundations)

Workshop ia-claude · alineada a los 5 dominios oficiales

raul.alzate@sofka.com.co

Índice

- Cómo usar esta guía (TL;DR)
- Examen de un vistazo
  - Formato
  - Dominios y peso
  - Los 6 escenarios
- Mapa escenario → kata
  - Mapa temático — los 30 katas por hilo
- Domain 1 — Agentic Architecture & Orchestration (27 %)
  - Task Statement 1.1: Agentic loops
  - Task Statement 1.2: Coordinador-subagente (hub-and-spoke)
  - Task Statement 1.3: Spawning de subagentes (Agent SDK)
  - Task Statement 1.4: Workflows multi-paso con enforcement
  - Task Statement 1.5: Hooks de Agent SDK
  - Task Statement 1.6: Task decomposition
  - Task Statement 1.7: Session management
- Domain 2 — Tool Design & MCP Integration (18 %)
  - Task Statement 2.1: Descripciones de tools
  - Task Statement 2.2: Errores estructurados (MCP)
  - Task Statement 2.3: Distribución de tools y `tool_choice`
  - Task Statement 2.4: MCP servers
  - Task Statement 2.5: Built-in tools
- Domain 3 — Claude Code Configuration & Workflows (20 %)
  - Task Statement 3.1: CLAUDE.md hierarchy
  - Task Statement 3.2: Custom commands & skills
  - Task Statement 3.3: Path-scoped rules
  - Task Statement 3.4: Plan mode vs direct execution
  - Task Statement 3.5: Iterative refinement
  - Task Statement 3.6: CI/CD pipelines
- Domain 4 — Prompt Engineering & Structured Output (20 %)
  - Task Statement 4.1: Criterios explícitos
  - Task Statement 4.2: Few-shot
  - Task Statement 4.3: Structured output con tool_use
  - Task Statement 4.4: Validation + retry
  - Task Statement 4.5: Batch processing
  - Task Statement 4.6: Multi-pass review
- Domain 5 — Context Management & Reliability (15 %)
  - Task Statement 5.1: Conversation context
  - Task Statement 5.2: Escalación
  - Task Statement 5.3: Error propagation multi-agente
  - Task Statement 5.4: Context en codebases grandes
  - Task Statement 5.5: Human review workflows
  - Task Statement 5.6: Provenance
- Cards conceptuales por kata
- Kata 01 — Bucle Agéntico Determinista
  - Concepto
  - Por qué importa
  - Modelo mental
  - Ejemplo mínimo

Anti-patrón
Argumento de certificación
Auto-evaluación
Kata 02 — Guardarraíles Deterministas con `PreToolUse`
Concepto
Por qué importa
Modelo mental
Ejemplo mínimo
Anti-patrón
Argumento de certificación
Auto-evaluación
Kata 03 — Normalización con `PostToolUse`
Concepto
Por qué importa
Modelo mental
Ejemplo mínimo
Anti-patrón
Argumento de certificación
Auto-evaluación
Kata 04 — Aislamiento Estricto de Subagentes (Hub-and-Spoke)
Concepto
Por qué importa
Modelo mental
Ejemplo mínimo
Anti-patrón
Argumento de certificación
Auto-evaluación
Kata 05 — Extracción Estructurada Defensiva con JSON Schema
Concepto
Por qué importa
Modelo mental
Ejemplo mínimo
Anti-patrón
Argumento de certificación
Auto-evaluación
Kata 06 — Errores Estructurados en MCP
Concepto
Por qué importa
Modelo mental
Ejemplo mínimo
Anti-patrón
Argumento de certificación
Auto-evaluación
Kata 07 — Exploración Segura con Plan Mode
Concepto
Por qué importa
Modelo mental
Ejemplo mínimo
Anti-patrón
Argumento de certificación
Auto-evaluación

Kata 08 — Memoria Jerárquica con `CLAUDE.md`
- Concepto
- Por qué importa
- Modelo mental
- Ejemplo mínimo
- Anti-patrón
- Argumento de certificación
- Auto-evaluación

Kata 09 — Reglas Condicionales por Ruta
- Concepto
- Por qué importa
- Modelo mental
- Ejemplo mínimo
- Anti-patrón
- Argumento de certificación
- Auto-evaluación

Kata 10 — Optimización Económica con Prefix Caching
- Concepto
- Por qué importa
- Modelo mental
- Ejemplo mínimo
- Anti-patrón
- Argumento de certificación
- Auto-evaluación

Kata 11 — Mitigación de Dilución Softmax (Edge Placement + Compactación)
- Concepto
- Por qué importa
- Modelo mental
- Ejemplo mínimo
- Anti-patrón
- Argumento de certificación
- Auto-evaluación

Kata 12 — Prompt Chaining Multi-Pass
- Concepto
- Por qué importa
- Modelo mental
- Ejemplo mínimo
- Anti-patrón
- Argumento de certificación
- Auto-evaluación

Kata 13 — Code Review Headless en CI/CD
- Concepto
- Por qué importa
- Modelo mental
- Ejemplo mínimo
- Anti-patrón
- Argumento de certificación
- Auto-evaluación

Kata 14 — Few-Shot para Calibrar Bordes
- Concepto

Por qué importa
Modelo mental
Ejemplo mínimo
Anti-patrón
Argumento de certificación
Auto-evaluación
Kata 15 — Evaluación Crítica y Auto-Corrección
Concepto
Por qué importa
Modelo mental
Ejemplo mínimo
Anti-patrón
Argumento de certificación
Auto-evaluación
Kata 16 — Protocolo de Handoff a Humano
Concepto
Por qué importa
Modelo mental
Ejemplo mínimo
Anti-patrón
Argumento de certificación
Auto-evaluación
Kata 17 — Procesamiento Masivo con Message Batches API
Concepto
Por qué importa
Modelo mental
Ejemplo mínimo
Anti-patrón
Argumento de certificación
Auto-evaluación
Kata 18 — Scratchpad Persistente
Concepto
Por qué importa
Modelo mental
Ejemplo mínimo
Anti-patrón
Argumento de certificación
Auto-evaluación
Kata 19 — Investigación Adaptativa (Descomposición Dinámica)
Concepto
Por qué importa
Modelo mental
Ejemplo mínimo
Anti-patrón
Argumento de certificación
Auto-evaluación
Kata 20 — Preservación de Provenance
Concepto
Por qué importa
Modelo mental
Ejemplo mínimo

Anti-patrón
Argumento de certificación
Auto-evaluación
Kata 21 — Calidad de Descripciones de Tools
Concepto
Por qué importa
Modelo mental
Ejemplo mínimo
Anti-patrón
Argumento de certificación
Auto-evaluación
Kata 22 — Configuración de MCP Servers
Concepto
Por qué importa
Modelo mental
Ejemplo mínimo
Anti-patrón
Argumento de certificación
Auto-evaluación
Kata 23 — Selección de Built-in Tools
Concepto
Por qué importa
Modelo mental
Ejemplo mínimo
Anti-patrón
Argumento de certificación
Auto-evaluación
Kata 24 — Slash Commands Custom y Skills
Concepto
Por qué importa
Modelo mental
Ejemplo mínimo
Anti-patrón
Argumento de certificación
Auto-evaluación
Kata 25 — Gestión de Sesiones (Resume y Fork)
Concepto
Por qué importa
Modelo mental
Ejemplo mínimo
Anti-patrón
Argumento de certificación
Auto-evaluación
Kata 26 — Validación y Retry con Error Feedback
Concepto
Por qué importa
Modelo mental
Ejemplo mínimo
Anti-patrón
Argumento de certificación

Auto-evaluación
Kata 27 — Multi-Pass Review e Independent Reviewer
Concepto
Por qué importa
Modelo mental
Ejemplo mínimo
Anti-patrón
Argumento de certificación
Auto-evaluación
Kata 28 — Propagación de Errores Multi-Agente
Concepto
Por qué importa
Modelo mental
Ejemplo mínimo
Anti-patrón
Argumento de certificación
Auto-evaluación
Kata 29 — Confidence Calibration y Stratified Sampling
Concepto
Por qué importa
Modelo mental
Ejemplo mínimo
Anti-patrón
Argumento de certificación
Auto-evaluación
Kata 30 — Criterios Explícitos para Reducir Falsos Positivos
Concepto
Por qué importa
Modelo mental
Ejemplo mínimo
Anti-patrón
Argumento de certificación
Auto-evaluación
Banco de preguntas de práctica
Q1 (Domain 1 / Kata 01)
Q2 (Domain 1 / Kata 02)
Q3 (Domain 1 / Kata 04)
Q4 (Domain 2 / Kata 02 — Tool descriptions)
Q5 (Domain 2 / Kata 06 — MCP errors)
Q6 (Domain 2 / Kata 16 — tool_choice)
Q7 (Domain 3 / Kata 08 — CLAUDE.md hierarchy)
Q8 (Domain 3 / Kata 09 — Path-scoped rules)
Q9 (Domain 3 / Kata 07 — Plan mode vs direct)
Q10 (Domain 3 / Custom slash command)
Q11 (Domain 4 / Kata 05 — Schema design)
Q12 (Domain 4 / Kata 14 — Few-shot)
Q13 (Domain 4 / Kata 15 — Validation/retry)
Q14 (Domain 4 / Kata 17 — Batch API)
Q15 (Domain 4 / Kata 13 — Multi-pass review)
Q16 (Domain 5 / Kata 11 — Context management)
Q17 (Domain 5 / Kata 16 — Escalación)

- Q18 (Domain 5 / Kata 06 — Error propagation multi-agente)
- Q19 (Domain 5 / Kata 20 — Provenance)
- Q20 (Domain 5 / Kata 18 — Scratchpad)

Topics bonus — examen-relevantes pero no profundos en katas

- Session resumption y forking (Agent SDK / Claude Code CLI)
- Skills frontmatter (Claude Code)
- Comandos de Claude Code
- Tools built-in vs MCP — cuándo cada uno
- Confidence calibration y stratified sampling
- Coverage gap reporting (Multi-agent)

Glosario

Cheat sheet (una página)

- Diez reglas para repetir antes del examen
- Diez anti-patrones para reconocer al instante

Out-of-scope (NO entra en el examen)

Cómo usar esta guía (TL;DR)

Esta guía está organizada para **prepararte directamente al examen oficial**. Tiene tres usos:

1. **Estudio dirigido por dominio.** Las páginas siguientes recorren los 5 dominios oficiales con sus *task statements* y los katas/notebooks que los cubren. Lee primero esto y identifica qué dominio te cuesta más.
2. **Práctica con preguntas.** El **Banco de preguntas** al final tiene ~20 preguntas multiple-choice en el formato exacto del examen (escenario + 4 opciones + explicación de por qué cada distractor falla).
3. **Repaso visual.** Las **cards conceptuales** (una por kata) son resúmenes de una página para hojear el día anterior.

> **Cero mocks.** Cada concepto está respaldado por un notebook ejecutable en `katas/kata_NNN_<slug>/notebook.ipynb`. La guía es la teoría; el notebook es la práctica.

Examen de un vistazo

Formato

Atributo	Valor
Tipo	Multiple-choice (1 correcta, 3 distractores)
Escenarios	4 de 6 escenarios fijos, escogidos al azar
Score scale	100–1000 (mínimo aprobación: **720**)
Penalización	No hay; preguntas en blanco cuentan como incorrectas

Dominios y peso

Dominio	Peso	Hilo principal
1. Agentic Architecture & Orchestration	27%	Bucles, hooks, subagentes
2. Tool Design & MCP Integration	18%	Tools, errores tipados, MCP
3. Claude Code Configuration & Workflows	20%	CLAUDE.md, skills, plan mode
4. Prompt Engineering & Structured Output	20%	Schemas, few-shot, batch
5. Context Management & Reliability	15%	Contexto, escalación, prov.

> **Lectura estratégica**: Dominios 1+3+4 = **67 % del examen**. Si tu tiempo es limitado, gasta el grueso ahí.

Los 6 escenarios

El examen presenta 4 de estos 6 al azar. **Conoce los 6** porque las preguntas son escenario-aware: una mala lectura del contexto cambia la respuesta correcta.

1. **Customer Support Resolution Agent** — Agent SDK; tools `get_customer`, `lookup_order`, `process_refund`, `escalate_to_human`; meta 80%+ de first-contact resolution.
2. **Code Generation with Claude Code** — Claude Code; CLAUDE.md, slash commands, plan mode vs direct execution.
3. **Multi-Agent Research System** — coordinador + subagentes (web search, document analysis, synthesis, report).
4. **Developer Productivity with Claude** — built-in tools (Read, Write, Bash, Grep, Glob), exploración de codebases, MCP servers.
5. **Claude Code for Continuous Integration** — `claude -p`, `--output-format json`, anotaciones inline en PRs.
6. **Structured Data Extraction** — `tool_use` con JSON Schema, validación, manejo de edge cases, downstream integration.

Mapa escenario → kata

Para cada escenario oficial, los katas (de los 30) que lo cubren directamente:

Escenario	Katas relevantes
Customer Support Agent	01, 02, 06, 15, 16, 21, 22, 25, 28, 30
Code Generation con Claude Code	07, 08, 09, 19, 23, 24, 25, 27
Multi-Agent Research	04, 12, 19, 20, 21, 28, 29
Developer Productivity	07, 18, 19, 22, 23, 24, 25
CI/CD Headless	13, 17, 26, 27, 30
Structured Data Extraction	05, 14, 15, 17, 26, 29, 30

> **Tip de examen**: cuando te toque un escenario, recorre primero los tres katas más asociados antes de mirar las opciones. Los distractores a menudo “huelen bien” si los lees en abstracto.

Mapa temático — los 30 katas por hilo

Una decisión técnica suele tocar varios hilos a la vez. Las conexiones inter-katas aparecen también al final de cada sección “Argumento de certificación” del kata.

Hilo	Katas
Determinismo	01, 02, 06, 13
Tools & MCP	02, 03, 06, 21, 22, 23, 24
Schemas y contratos	05, 13, 15, 16, 20, 26, 29
Economía de contexto	03, 08, 09, 10, 11, 12
Memoria y sesión	08, 11, 18, 19, 25
Aislamiento y subagentes	04, 12, 19, 28
Human-in-the-loop	07, 15, 16, 30
Calidad pragmática (false pos.)	13, 14, 27, 30
Provenance y confianza	15, 20, 29

Domain 1 — Agentic Architecture & Orchestration (27 %)

Task Statement 1.1: Agentic loops

Lo que tienes que saber con tus dedos:

```
# Patrón canónico (cero mocks; ver Kata 01)
while True:
    resp = client.messages.create(messages=history, tools=tools, ...)
    if resp.stop_reason == "tool_use":
        result = run_tool(resp.content)            # ejecutar
        history.append({"role": "assistant", "content": resp.content})
        history.append({"role": "user", "content": [
            {"type": "tool_result", "tool_use_id": tu.id, "content": str(result)}
        ]})
        continue
    if resp.stop_reason == "end_turn":
        return resp                                # éxito
    raise UnhandledStop(resp.stop_reason)          # nunca silencioso
```

Anti-patrones que el examen ataca:

- Parsear texto del modelo para decidir terminación (`if "done" in text`).
- Usar `max_iterations` como mecanismo primario de corte.
- Inferir éxito de la ausencia de errores.

> Notebook de referencia: `kata_001_agentic_loop`.

Task Statement 1.2: Coordinador-subagente (hub-and-spoke)

Reglas duras:

- El coordinador es el **único** que recibe la query final del usuario.
- Cada subagente es una **sesión nueva** con prompt mínimo y tools recortadas.
- Subagentes devuelven **JSON tipado**, jamás prosa libre.
- La memoria conversacional **NO se hereda automáticamente**.

```
def coordinator(user_query):
    plan = decompose(user_query)                   # subtareas tipadas
    results = [run_subagent(t) for t in plan]      # cada uno: sesión nueva
    return synthesize(results)                     # agrega claims tipados

def run_subagent(subtask):
    return client.messages.create(
        system=focused_system_for(subtask),        # prompt MÍNIMO
        messages=[{"role":"user","content": subtask.prompt}],
        tools=subtask.scoped_tools,                # subset acotado
        tool_choice={"type":"any"},                # fuerza salida tipada
    )
```

> Notebooks: `kata_004_subagent_isolation`, `kata_019_adaptive_investigation`.

Task Statement 1.3: Spawning de subagentes (Agent SDK)

Términos del SDK que aparecen en el examen:

- **`Task` tool**: el mecanismo del Agent SDK para que el coordinador invoque subagentes. Para que esto funcione, `allowedTools` del coordinador **debe** incluir `"Task"`.
- **`AgentDefinition`**: configuración por subagente — descripción, system prompt y tool restrictions. Cada tipo de subagente puede tener su propio `AgentDefinition`.

- **Paralelismo**: el coordinador emite **múltiples `Task` calls en una sola respuesta** para que corran en paralelo. Calls en turnos separados son secuenciales.

> Punto de examen: si un subagente "olvida" el contexto del coordinador, es porque NO lo hereda automáticamente — hay que pasarlo explícito en su prompt.

Task Statement 1.4: Workflows multi-paso con enforcement

Programmatic enforcement vs prompt-based:

Necesidad	Mecanismo
Compliance crítico (financiero)	Hook que bloquea hasta cumplir prereq.
Workflow ordering "preferido"	System prompt o few-shot

Patrón "prerequisite gate":

```
def pretool_gate(tool_name, tool_input, session_state):
    if tool_name == "process_refund" and not session_state.customer_verified:
        return {"action":"deny","reason":"prereq:get_customer required"}
    return {"action":"allow"}
```

> El examen mete preguntas con frase "production data shows that in 12 % of cases…" para forzarte a elegir hooks (deterministic) sobre prompt.

Task Statement 1.5: Hooks de Agent SDK

- **`PostToolUse`**: normaliza el resultado **antes** de que el modelo lo vea (Kata 03). Caso típico: timestamps Unix → ISO 8601, status codes → strings legibles.
- **Tool call interception**: equivalente del `PreToolUse` (Kata 02). Bloquea por política, redirige a escalación.

Task Statement 1.6: Task decomposition

- **Prompt chaining** (Kata 12): pasos *fijos* y *secuenciales*. Apto cuando el flujo es predecible (analizar 50 archivos, luego integrar).
- **Adaptive decomposition** (Kata 19): generar el plan al vuelo cuando el problema es exploratorio (codebase desconocido).

Task Statement 1.7: Session management

Comando / función	Cuándo usar
`--resume <name>`	Continuar sesión nombrada (contexto válido)
`fork_session`	Explorar dos enfoques desde una baseline
Sesión nueva + summary	Cuando los tool results previos están stale

> Notebooks: `kata_018_scratchpad_persistence` cubre el patrón de summary persistente que sobrevive al `/compact`.

Domain 2 — Tool Design & MCP Integration (18 %)

Task Statement 2.1: Descripciones de tools

El error #1 que el examen detecta: tools con descripciones genéricas que se solapan.

Mal	Bien
`analyze_content` : “Analyzes content”	`extract_web_results` : “Parses HTML…”
`analyze_document` : “Analyzes documents”	`verify_claim_against_source` : “Validates…”

Receta: descripción debe incluir **input format**, **example queries**, **edge cases**, **boundary** (cuándo usar este vs un similar).

Task Statement 2.2: Errores estructurados (MCP)

```
{
  "isError": true,
  "errorCategory": "rate_limit",        // transient | validation | business | permission
  "isRetryable": true,
  "retryAfterSeconds": 30,
  "explanation": "API quota exceeded for tenant X"
}
```

Reglas:

- `errorCategory` distingue 4 familias; cada una tiene política de retry distinta.
- `isRetryable: false` para business/permission errors → el agente **no** debe reintentar.
- **Local recovery primero**: el subagente reintenta transient localmente y sólo propaga al coordinador lo que no pudo resolver.

Task Statement 2.3: Distribución de tools y `tool_choice`

`tool_choice`	Significado
`{"type": "auto"}` (default)	Modelo decide; puede no llamar tool
`{"type": "any"}`	DEBE llamar algún tool
`{"type": "tool", "name": "X"}`	DEBE llamar exactamente ese tool

Anti-patrón #1: dar 18 tools a un agente. **Regla del 4-5**: cada agente debe tener tools del tamaño de un dedo de la mano.

Task Statement 2.4: MCP servers

Scope	Path	Compartido
Project	`.mcp.json`	Sí (git)
User	`~/.claude.json`	No

```
// .mcp.json — credenciales por env var, NO hardcoded
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["-y","@modelcontextprotocol/server-github"],
      "env": {"GITHUB_TOKEN": "${GITHUB_TOKEN}"}
    }
  }
}
```

Task Statement 2.5: Built-in tools

Tool	Para qué
Grep	Buscar contenido (regex en archivos)
Glob	Buscar archivos por patrón de path (`**/*.test.tsx`)
Read	Cargar archivo completo
Edit	Modificación dirigida con anchor único
Write	Sobrescribir; fallback cuando Edit no encuentra anchor
Bash	Comandos shell

Anti-patrón: leer todos los archivos al inicio. Estrategia correcta: Grep para hallar entry points → Read para seguir imports → Edit/Write puntual.

Domain 3 — Claude Code Configuration & Workflows (20 %)

Task Statement 3.1: CLAUDE.md hierarchy

Nivel	Path	Compartido
User	`~/.claude/CLAUDE.md`	No
Project	`.claude/CLAUDE.md` o `CLAUDE.md`	Sí
Directory	`<subdir>/CLAUDE.md`	Sí

```
# CLAUDE.md raíz
@docs/coding-standards.md          # @import modular
@docs/testing.md
```

`@import` vs `.claude/rules/`: ambos valen. `@import` es bueno para reutilizar standards entre paquetes. `.claude/rules/` es mejor cuando quieres reglas condicionales por path (ver 3.3).

> Comando útil: `/memory` muestra qué archivos están cargados en la sesión actual.

Task Statement 3.2: Custom commands & skills

Artefacto	Path	Frontmatter clave
Slash command (proyecto)	`.claude/commands/X.md`	(ninguno requerido)
Slash command (personal)	`~/.claude/commands/X.md`	
Skill (proyecto)	`.claude/skills/X/SKILL.md`	`context`, `allowed-tools`, `argument-hint`

```
---
name: codebase-analysis
context: fork                      # corre en sub-agente aislado
allowed-tools: ["Read","Grep","Glob"]   # NO Write/Bash
argument-hint: "<file-or-dir>"
---
# Skill body
Analiza la estructura de {{argument}}…
```

`context: fork` = la skill no contamina el main session con su output verbose. Crítico para skills exploratorias.

Task Statement 3.3: Path-scoped rules

```
# .claude/rules/tests.md
---
paths: ["**/*.test.tsx", "**/*.test.ts"]
---

# Convenciones de tests
- Usa Vitest, no Jest.
- AAA pattern: arrange/act/assert.
```

Cuándo glob-rules superan a CLAUDE.md de subdirectorio: cuando los archivos a los que aplica la regla están **dispersos por todo el repo** (p. ej., tests al lado del código). Subdirectorio sólo cubre un árbol.

Task Statement 3.4: Plan mode vs direct execution

Caso	Modo
Refactor monolito → microservicios	**Plan mode**
Bug fix en una función con stack trace	**Direct execution**
Migración de librería que toca 45+ archivos	**Plan mode**
Añadir validación de fecha a 1 endpoint	**Direct execution**

> Heurística: si hay **decisiones arquitectónicas** o **caminos alternativos**, plan mode. Si hay un patch claro de scope acotado, direct.

Task Statement 3.5: Iterative refinement

Patrones del examen:

- **Few-shot examples** (2-4) cuando la prosa no comunica el formato.
- **Test-driven**: escribir tests primero, iterar con failures.
- **Interview pattern**: que Claude haga preguntas antes de implementar.
- **Sequential vs single-message**: si los problemas interactúan, mensaje único con todos los issues; si son independientes, secuencial.

Task Statement 3.6: CI/CD pipelines

```
claude -p "$REVIEW_PROMPT"        --output-format json        --json-schema annotations.schema.json
```

Reglas:

- `-p` evita hangs en CI (no espera input interactivo).
- `--output-format json` + `--json-schema` produce salida machine-parseable.
- **Sesión independiente para review**: la misma sesión que generó el código es **peor** revisora que una instancia limpia.

Domain 4 — Prompt Engineering & Structured Output (20 %)

Task Statement 4.1: Criterios explícitos

Vago	Explícito
“be conservative with findings”	“report issues only when actual code behavior contradicts comments”
“only report high-confidence issues”	“severity ≥ medium **AND** matches one of these 5 categories”

> Falsos positivos altos en una categoría matan la confianza en TODAS. A veces conviene**deshabilitar** temporalmente la categoría problemática.

Task Statement 4.2: Few-shot

- **2-4 ejemplos** de bordes, no de centro fácil.
- Mostrar **razonamiento** (por qué se eligió tal opción) en escenarios ambiguos.
- Cubrir **formatos variados** (citas inline vs bibliografía, narrativo vs tabla) cuando el documento puede venir en varias formas.

Task Statement 4.3: Structured output con tool_use

```
EXTRACT = {
    "name": "extract_invoice",
    "input_schema": {
        "type": "object",
        "properties": {
            "invoice_id": {"type": "string"},
            "stated_total": {"type": "number"},
            "computed_total": {"type": "number"},
            "currency": {"type": "string", "enum": ["USD","EUR","COP","other"]},
            "currency_other_details": {"type": ["string","null"]},
            "status": {"type": "string", "enum": ["paid","pending","unclear"]},
            "due_date": {"type": ["string","null"], "format": "date"},
        },
        "required": ["invoice_id","stated_total","computed_total","currency","status"]
    }
}
resp = client.messages.create(
    tools=[EXTRACT],
    tool_choice={"type": "tool", "name": "extract_invoice"},
    ...
)
```

Diseño defensivo:

- `enum` cerrado **+** `"other"` con detail string para extensibilidad.
- `null` en lugar de string vacío cuando el dato no existe.
- `tool_use` elimina errores de **sintaxis** JSON; **no** elimina errores semánticos (sumas que no cuadran).

Task Statement 4.4: Validation + retry

Loop de auto-corrección:

```python
def extract_with_retry(doc, attempts=2):
    extraction = extract(doc)
    error = pydantic_validate(extraction)
    if not error: return extraction
    for _ in range(attempts):
        extraction = extract(doc, error_feedback=error)
        error = pydantic_validate(extraction)
        if not error: return extraction
    return {"extraction": extraction, "validation_error": error}
```

Cuándo retry NO ayuda: el dato simplemente NO está en el documento. Reintentar es alucinación garantizada — mejor `null` + `needs_human_review`.

Task Statement 4.5: Batch processing

Caso	API
Pre-merge checks (devs esperando)	Synchronous (real-time)
Reportes overnight, audits semanales	**Message Batches** (50% más barato)

Restricciones del Batch API:

- Hasta 24 h de ventana, sin SLA garantizada.
- **NO soporta multi-turn tool calling** dentro de un request — batch es para llamadas de un solo turno.
- `custom_id` correlaciona request↔response; resubmissions sólo de los fallidos.

Task Statement 4.6: Multi-pass review

- **Self-review limitations**: el modelo que generó código retiene contexto de razonamiento — es peor revisor de su propio output.
- **Independent instance**: una instancia limpia (sin la cadena de generación) detecta más issues.
- **Per-file pass + cross-file integration pass** para PRs grandes; evita attention dilution.

Domain 5 — Context Management & Reliability (15 %)

Task Statement 5.1: Conversation context

Riesgos de la summarización progresiva:

- Valores numéricos (montos, fechas, customer-stated expectations) se comprimen a frases vagas.
- “Lost in the middle”: atención U-shape, hallazgos del centro se diluyen.

Patrón “case facts”: extraer datos transaccionales (`order_id`, `amount`, `status`) a un bloque persistente que se incluye **en cada prompt**, fuera del summary.

Task Statement 5.2: Escalación

Triggers válidos:

- Cliente pide explícitamente un humano (honra inmediato).
- Policy gap (la política no cubre el caso).
- Inability to make meaningful progress.

Triggers inválidos (que el examen ataca):

- Sentiment-based (“cliente parece molesto”) — no correlaciona con complejidad.
- Self-reported confidence score del LLM — está mal calibrado.

Caso “multiple matches”: si una búsqueda devuelve N resultados, **pide un identificador adicional** al cliente, no escojas heurísticamente.

Task Statement 5.3: Error propagation multi-agente

Patrón correcto	Anti-patrón
Subagente: error tipado al coordinador	Suprimir error como “success vacío”
`{failure_type, attempted, partial}`	`"search unavailable"` genérico
Retry transients localmente	Propagar todo error al coordinador
Annotaciones de coverage gap en synth	Terminar workflow al primer error

Task Statement 5.4: Context en codebases grandes

- **Subagent delegation**: el main agent coordina; los subagentes exploran (ver Explore subagent).
- **Scratchpad files**: hallazgos durables sobreviven `/compact`.
- **Summary-injected new session** > resumir 60 mensajes de history que contiene tool results stale.

Task Statement 5.5: Human review workflows

- **Stratified random sampling** sobre extracciones high-confidence para detectar nuevos patrones de error.
- **Field-level confidence scores** calibrados con un validation set etiquetado (no la confianza self-reported del modelo).
- Aggregate “97 % accuracy” puede ocultar mal performance en un tipo de doc → mide por tipo y campo.

Task Statement 5.6: Provenance

- Cada claim con `source_id`, `source_name`, `publication_date`, `relevant_excerpt`.
- **Conflictos preservados, no resueltos**: dos sources con stats distintas → registrar ambos con anotación, no promediar.
- **Fechas explícitas** evitan que diferencias temporales legítimas se lean como contradicciones.

Cards conceptuales por kata

Una página por kata. Útil como repaso visual el día anterior.

Kata 01 — Bucle Agéntico Determinista

Concepto

Un agente Claude no termina cuando "dice" que terminó: termina cuando la API devuelve un `stop_reason` terminal. El bucle agéntico es un `while` controlado por metadatos estructurados (`stop_reason`), no por el contenido del texto que genera el modelo.

Tres ramas posibles por turno:

- `tool_use` → ejecutar la herramienta, anexar el resultado al historial, iterar.
- `end_turn` → detener el bucle, devolver respuesta final.
- cualquier otro (`max_tokens`, ausente, desconocido) → detener con motivo explícito; nunca seguir adivinando.

Por qué importa

El primer error de un sistema agéntico es decidir terminación con `if "done" in text`. El día que el modelo escribe "we are done" en mitad de una cadena de herramientas, el agente corta antes de tiempo y el negocio paga la diferencia. La certificación exige que el control de flujo sea **observable y reproducible**; sólo `stop_reason` cumple eso.

Modelo mental

- El texto del modelo es **carga útil**, no señal de control.
- El bucle es un autómata: estado = historial; transición = `stop_reason`.
- Cada iteración deja una entrada en un log estructurado (índice, señal, rama, herramienta, motivo de fin) suficiente para reproducir la traza sin re-ejecutar.

Ejemplo mínimo

```
while True:
    resp = client.messages.create(messages=history, tools=tools, ...)
    if resp.stop_reason == "tool_use":
        result = run_tool(resp.content)            # determinista
        history.append(result)
        continue
    if resp.stop_reason == "end_turn":
        return resp                                # éxito
    raise UnhandledStop(resp.stop_reason)          # nunca silencioso
```

Ningún `re.search`, ningún `"done" in text`. Sólo metadatos.

Anti-patrón

Decidir terminación, despacho de herramientas o reintentos a partir del texto generado (regex, `in`, `contains`). Falla silenciosamente cuando el modelo parafrasea, idioma cambia, o el usuario inyecta la frase trampa. Es la forma más común de "agente que casi funciona en demo y se rompe en producción".

Argumento de certificación

- Sé explicar por qué `stop_reason` es la única fuente legítima de control.
- Sé enumerar los `stop_reason` posibles y la respuesta correcta para cada uno.
- Sé describir el log mínimo que hace una corrida auditable y reproducible.

Auto-evaluación

1. ¿Qué ocurre si el modelo devuelve `stop_reason=max_tokens`? ¿Continúo, abortó, reintento?

Aborto con motivo explícito (`unhandled:max_tokens`), como muestra la celda §3.4. No reintento automático: la respuesta está truncada, así que reintentar sin cambiar nada repetiría el truncamiento. Lo correcto es subir el cap de `max_tokens` para esa llamada, o pedir al modelo que continúe en un turno nuevo (turno explícito iniciado por el cliente, no por el bucle). El bucle deja la decisión al llamador y registra la causa en el log.

2. Si una herramienta lanza excepción, ¿cómo lo veo en el historial sin romper el invariante "control por señal"?

Capturo la excepción dentro del bloque `tool_use`, la convierto en un `tool_result` con campo `error`, y la anexo al historial igual que cualquier otro resultado (en `run_loop_signal` lo hago en el bloque `try/except`). El bucle continúa: la próxima iteración mostrará al modelo el error tipado y le permitirá replanificar. El control sigue siendo `stop_reason`; el fallo es payload, no señal.

3. ¿Qué información mínima debe registrar el log para reconstruir la traza completa sin volver a llamar al modelo?

Por iteración: índice, `stop_reason`, rama tomada (`dispatch` / `halt`), nombre de la herramienta cuando aplique, status (`ok` / `error`) de la ejecución y, en la iteración terminal, la causa de corte (`end_turn` o `unhandled:<reason>`). Es exactamente lo que produce `Logger.show()`. Para replicar la respuesta del modelo bit-a-bit haría falta también el guion crudo, pero el log de control basta para auditar la lógica del agente.

Kata 02 — Guardarraíles Deterministas con `PreToolUse`

Concepto

Un hook `PreToolUse` se ejecuta **antes** de que la herramienta corra. Puede permitir, denegar o pedir aprobación humana basándose en el payload exacto del `tool_use`. La política se aplica en código, no en el prompt.

Por qué importa

Pedirle al modelo "no borres datos de producción" en el system prompt es una sugerencia, no un control. Un `PreToolUse` que rechaza `DROP TABLE` o `rm -rf /` es un control verificable. La diferencia define si el sistema es auditable o sólo "casi seguro".

Modelo mental

- El prompt **sugiere**; el hook **aplica**.
- Veredictos tipados: `allow | deny | ask_human`, siempre con razón.
- La política vive en archivo (JSON/YAML) recargable en caliente. El modelo no la ve para que no pueda persuadirla.
- Cada veredicto produce evento estructurado: herramienta, args, decisión, regla.

Ejemplo mínimo

```
def pretool_hook(tool_name, tool_input) -> Verdict:
    if tool_name == "shell" and re.search(r"\brm -rf\b", tool_input["cmd"]):
        return Verdict("deny", reason="POL-DELETE-001")
    if tool_name == "refund" and tool_input["amount"] > 1000:
        return Verdict("ask_human", reason="POL-REFUND-LIMIT")
    return Verdict("allow")
```

Si el hook deniega, el modelo recibe un `tool_result` con error tipado y puede replanificar. La acción peligrosa nunca se ejecuta.

Anti-patrón

"Defensa en prompt": párrafos en el system prompt rogando al modelo no hacer X. Funciona el 95 % del tiempo y falla justo cuando importa: jailbreak, prompt injection en datos del usuario, comportamiento emergente de un tool nuevo.

Argumento de certificación

- Sé distinguir control suave (prompt) de control duro (hook).
- Sé describir los tres veredictos y cuándo emitir cada uno.
- Sé justificar por qué la política vive fuera del prompt y se recarga en caliente.

Auto-evaluación

1. Si la política cambia mientras hay sesión activa, ¿cómo aseguro que el hook use la versión nueva sin reiniciar el agente?

`pretool_check` lee `POLICY` cada vez que se invoca. Si reemplazo el dict (o lo recargo desde JSON), la siguiente llamada al hook ya ve la nueva versión. La sesión del modelo no necesita reiniciarse — el hook es el único que conoce la política y vive del lado del cliente.

2. ¿Cómo distingo "denegar y dejar continuar" de "denegar y escalar"?

Son dos veredictos distintos. `deny` produce un `tool_result` con `error_code`; el modelo lo recibe como contexto y

replanifica (puede decirle al usuario que no se puede). `ask_human` produce un `tool_result` con `requires_approval=true` y, en producción, despacha a la cola humana (ver Kata 16).

3. ¿Qué prueba reintroduce el anti-patrón a propósito para verificar que el hook lo bloquea?

La celda §4 con el ataque social (CFO de emergencia). Si el bucle sin gate ejecuta el reembolso, demuestra que el prompt-only falla; si el bucle con gate registra `verdict=deny`, demuestra que la política aplicada externamente sigue siendo robusta.

Kata 03 — Normalización con `PostToolUse`

Concepto

Un hook `PostToolUse` corre **después** de la herramienta y **antes** de que el resultado llegue al contexto del modelo. Limpia, traduce códigos arcanos y proyecta el payload a un JSON mínimo y predecible.

Por qué importa

Si la herramienta devuelve XML legacy de 5 KB con códigos `0xA7`, el modelo gasta tokens para parsearlo cada turno y, peor, alucina cuando el formato cambia. Normalizar una vez, en un sitio determinista, ahorra contexto y elimina ambigüedad.

Modelo mental

- El modelo no debería ver formatos crudos jamás. Sólo JSON canónico.
- Los códigos numéricos/legacy se mapean a strings legibles (`0xA7` → `"timeout"`).
- El hook es puro: misma entrada → mismo JSON. Probable de testear sin LLM.
- Mide en tokens: el run normalizado debe ser estrictamente más barato que el crudo.

Ejemplo mínimo

```
def posttool_hook(tool_name, raw) -> dict:
    if tool_name != "legacy_lookup":
        return raw
    return {
        "id": raw["@id"],
        "status": STATUS_CODE_MAP.get(raw["s"], "unknown"),
        "amount": float(raw["amt"]),
    }
```

El historial almacena el JSON limpio; el XML original se descarta del contexto.

Anti-patrón

Inyectar el payload crudo y “que el modelo se las arregle”. Cada turno re-paga los tokens del parseo y la primera vez que el proveedor cambia el formato (o añade un código) el agente alucina o falla en silencio.

Argumento de certificación

- Sé describir cuándo `PostToolUse` aplica vs `PreToolUse`.
- Sé enunciar la regla “el modelo nunca ve datos crudos heterogéneos”.
- Sé medir el ahorro de tokens y tasa de mala interpretación con/sin hook.

Auto-evaluación

1. ¿Qué pasa si el hook recibe un código que no está en el mapa?

Devuelvo `"unknown"` para no romper el contrato (el campo siempre existe). En producción, además, registro un evento de “código no mapeado” para que el equipo lo revise. Nunca dejo que el modelo “adivine” qué significa.

2. ¿Cómo pruebo el hook sin levantar el modelo?

`posttool_normalize` es una función pura sobre dicts. Le paso un fixture y compruebo el resultado con `assert`. Cero llamadas a la API. Si más adelante el formato legacy cambia, agrego entradas al fixture y al `STATUS_MAP`.

3. ¿Qué métrica concreta demuestra que el hook redujo “carga cognitiva”?

`response.usage.input_tokens` en la segunda llamada (la que ve el `tool_result`). Comparé el run normalizado vs el crudo en §4. Para evidencia más fuerte, podría correr un eval con N invocaciones y medir tasa de respuestas correctas.

Kata 04 — Aislamiento Estricto de Subagentes (Hub-and-Spoke)

Concepto

Un coordinador descompone una tarea grande y lanza subagentes. Cada subagente recibe **sólo** el prompt mínimo que necesita y devuelve **sólo** un JSON tipado. Cero memoria compartida, cero historial heredado.

Por qué importa

Heredar el historial completo del coordinador parece "ayudar al subagente a entender el contexto". En la práctica diluye su atención, filtra políticas sensibles, y hace los fallos imposibles de bisecar (¿de qué turno ancestral viene este sesgo?).

Modelo mental

- Coordinador = hub. Subagentes = spokes. No hay aristas spoke↔spoke.
- Contrato de entrada del subagente: prompt + payload tipado, nada más.
- Contrato de salida: schema JSON. Si no encaja, error de boundary, no "casi".
- El coordinador es el único que agrega resultados.

Ejemplo mínimo

```
def coordinator(task):
    chunks = split(task)
    results = [run_subagent(prompt=focused_prompt(c), schema=ChunkResult)
               for c in chunks]
    return aggregate(results)            # JSONs tipados, no prosa
```

`run_subagent` arranca sesión nueva, sin pasar `coordinator.history`.

Anti-patrón

"Telepatía compartida": pasar todo el historial del coordinador al subagente "por si acaso". Resultado: contexto diluido, atención degradada, política filtrada a un agente que no tenía por qué verla.

Argumento de certificación

- Sé dibujar el diagrama hub-and-spoke y explicar por qué no hay aristas laterales.
- Sé enumerar qué se pasa al subagente y qué no.
- Sé argumentar por qué la salida tipada es preferible a "resúmen en prosa".

Auto-evaluación

1. ¿Cómo demuestro que el subagente NO recibió historial del coordinador?

`run_subagent` arma `messages=[{"role":"user", ...}]` desde cero por cada llamada. No hay variable `coordinator_history` que se pase. Si quisiera evidencia adicional: imprimir `len(messages)` antes de cada `create` — siempre debe ser 1.

2. ¿Qué hago si dos subagentes producen JSONs en conflicto?

Los registro ambos en el agregado (ver `coordinator_aggregate`: cada `value` viene con `source`). La resolución se delega: si el conflicto importa, escala a humano (Kata 16); si no, queda como dato preservado con provenance (Kata 20). Nunca elijo uno arbitrariamente.

3. ¿Cuándo está justificado romper el aislamiento?

Casi nunca. La excepción razonable es pasar artefactos compartidos read-only (un schema, una guía de estilo) en el system prompt — pero eso es contexto estable, no historial conversacional. Si me veo tentado a pasar history, primero pregunto si el problema se resuelve con un mejor prompt o partiendo distinto la tarea.

Kata 05 — Extracción Estructurada Defensiva con JSON Schema

Concepto

Para extraer datos de texto no estructurado, se fuerza al modelo a usar un **tool call con JSON Schema**. El schema declara: campos obligatorios reales, campos opcionales con uniones nullable, y enums con válvula de escape (`other`, `unclear`) acompañada de un campo `details`.

Por qué importa

Pedirle al modelo "devuélveme JSON" en prosa libre garantiza alucinación silenciosa: inventará campos faltantes, llenará vacíos con `""`, o forzará valores fuera de dominio. El schema es el único contrato que falla cerrado.

Modelo mental

- "Required" significa "siempre presente en la fuente"; si no lo es, márcalo nullable union.
- Default `""` es alucinación: si no sabe, debe ser `null` o `unclear`.
- Enums sin escape obligan al modelo a mentir cuando el valor no encaja.
- `tool_choice` forzado evita que devuelva texto y haga "best-effort prosa".

Ejemplo mínimo

```
{
  "name": "extract_invoice",
  "input_schema": {
    "type": "object",
    "required": ["invoice_id", "currency", "status"],
    "properties": {
      "invoice_id": {"type": "string"},
      "currency": {"type": "string", "enum": ["USD","EUR","COP","other"]},
      "currency_other_details": {"type": ["string","null"]},
      "status": {"type": "string", "enum": ["paid","pending","unclear"]},
      "due_date": {"type": ["string","null"], "format": "date"}
    }
  }
}
```

El cliente valida el resultado y rechaza si no encaja; nunca "ajusta" la salida.

Anti-patrón

JSON pedido en prosa, sin schema, con todos los campos `required` y sin enums con escape. Garantiza fabricación silenciosa: el modelo prefiere inventar a admitir incertidumbre.

Argumento de certificación

- Sé justificar `tool_choice` forzado vs prompt-only.
- Sé explicar la diferencia entre `nullable` y `optional` y por qué importa.
- Sé diseñar enums que admitan ambigüedad sin romper el contrato.

Auto-evaluación

1. Si el dato no aparece en la fuente, ¿qué valor debe poner el modelo?

`null` para campos nullable; el enum de escape (`other`, `unclear`) para enums. **Nunca** un valor inventado plausible.

2. ¿Por qué `category_other_details` es necesario al lado del enum?

Si elige `other`, necesito saber qué quiso decir. Sin el campo paralelo el `other` no aporta información — es básicamente un null. Con él, el agente registra la categoría no contemplada y yo decido si vale la pena agregarla al enum.

3. ¿Qué prueba demuestra que el schema bloquea fabricación cuando reintroduzco `required` excesivos?

Si marco `customer_id` como required, en los casos 2 y 4 el modelo se ve forzado a inventarlo. Comparar la salida del extractor con `customer_id: required` vs nullable demuestra el impacto. La aserción defensiva: pasar un texto sin id explícito y verificar que el extractor con schema correcto devuelve `null`.

Kata 06 — Errores Estructurados en MCP

Concepto

Un servidor MCP que falla NO devuelve un string genérico de error. Devuelve un payload tipado: `isError: true`, `errorCategory`, `isRetryable`, y `explanation`. El agente decide reintento, escalada o aborto a partir de esos campos, no del texto del error.

Por qué importa

`"something went wrong"` obliga al modelo a adivinar qué hacer: reintenta para siempre, abandona, o escala mal. Un error tipado convierte fallos en transiciones deterministas del bucle agéntico (Kata 1).

Modelo mental

- Tres ejes: ¿es error? ¿es reintentable? ¿qué categoría (auth, rate_limit, not_found, validation, transient)?
- El agente lee los flags, no la prosa.
- `explanation` es para el humano que audita el log, no para el modelo.
- Las políticas de retry (backoff, n máximo) se ejecutan en el cliente.

Ejemplo mínimo

```json
{
  "isError": true,
  "errorCategory": "rate_limit",
  "isRetryable": true,
  "retryAfterSeconds": 30,
  "explanation": "API quota exceeded for tenant X"
}
```

Cliente:

```python
if result.isError and result.isRetryable:
    sleep(result.retryAfterSeconds); continue
if result.isError and result.errorCategory == "auth":
    return escalate(result)
```

Anti-patrón

Devolver `raise Exception("failed")` o un string `"Error: ..."` y dejar que el modelo “razone” qué hacer. El agente entrará en bucle, abandonará, o escalará casos que sólo necesitaban backoff.

Argumento de certificación

- Sé enumerar las categorías de error MCP típicas y la respuesta correcta para cada una.
- Sé justificar por qué la decisión de retry vive en el cliente, no en el modelo.
- Sé describir cómo este kata se apoya en Kata 1 (control por señal) y prepara Kata 16 (escalada humana).

Auto-evaluación

1. ¿Cómo trato un error que llega sin `errorCategory`?

Lo trato como `errorCategory="unknown"` y `isRetryable=false` por defecto. Loguear y escalar es preferible a un retry ciego. La defensa es estricta: el cliente sólo confía en lo que el contrato declara.

2. ¿Cuál es la diferencia entre `transient` y `rate_limit` en política de retry?

`transient` es backoff exponencial corto (ms-segundos), `rate_limit` respeta `retryAfterSeconds` que viene del proveedor. Son curvas de espera distintas; la categoría se la doy al ejecutor de retry, no al modelo.

3. ¿Qué prueba reintroduce el anti-patrón (string genérico) y qué assert falla?

Llamar al handler `flaky_string` con el bucle robusto: el bucle no encuentra `isError` / `isRetryable`, asume éxito, y el modelo recibe un `tool_result` falso. Una aserción defensiva: validar el shape del result con un schema antes de pasarlo al historial; falla si llega un string crudo.

Kata 07 — Exploración Segura con Plan Mode

Concepto

Antes de modificar un repositorio desconocido, el agente entra en **Plan Mode**: modo sólo-lectura. Explora, escribe un documento markdown con hallazgos y arquitectura propuesta, y obtiene aprobación humana **antes** de transicionar a ejecución directa (escritura).

Por qué importa

Una refactorización masiva sin reconocimiento previo es destrucción probabilística. Plan Mode separa "entender" de "modificar"; el humano firma el plan, no cada edición.

Modelo mental

- Dos modos: read-only (Plan) y write (Direct). La transición es explícita y registrada.
- En Plan Mode, las herramientas de escritura están deshabilitadas, no sólo desaconsejadas.
- El artefacto de aprobación es texto auditable, no un "ok" verbal.
- Aprobación = firma + plan congelado; cambios al plan re-piden aprobación.

Ejemplo mínimo

```
1. /plan        # entra en read-only
2. agente lee, busca, mapea, redacta plan.md
3. humano revisa plan.md → "approve"
4. agente sale de Plan Mode → ejecuta sólo lo que el plan describe
```

Si el agente intenta escribir en Plan Mode, el hook `PreToolUse` lo deniega.

Anti-patrón

Lanzar el agente con permisos de escritura en un repo desconocido y "ver qué pasa". El primer error borra archivos clave o reescribe convenciones; recuperar es caro.

Argumento de certificación

- Sé describir qué herramientas están permitidas en cada modo.
- Sé justificar por qué la aprobación humana es sobre el plan, no sobre cada edición.
- Sé conectar Plan Mode con Kata 2 (los hooks aplican el modo) y Kata 16 (escalación cuando el plan se invalida).

Auto-evaluación

1. ¿Qué pasa si durante la ejecución el agente descubre que el plan era incorrecto?

Debería detenerse y volver a fase de plan. En esta implementación lo modelaría devolviendo un `tool_result` con `error_code: PLAN_INVALIDATED` y un nuevo turno donde el cliente regresa a Plan Mode (read-only) hasta que se apruebe un plan revisado.

2. ¿Cómo aseguro que Plan Mode bloquea TODA escritura, no sólo las "obvias"?

El cliente pasa explícitamente `tools=READ_ONLY_TOOLS`. No hay forma de que el modelo invoque `apply_patch`: el SDK rechaza tool_use con nombres que no están en la declaración. Adicionalmente, el bucle valida `tu.name in {t["name"] for t in tools}` antes de ejecutar.

3. ¿Dónde se almacena la aprobación humana para que la auditoría la encuentre?

En el log de eventos del bucle, con el plan markdown como adjunto. En producción, en una tabla

`plan_approvals(plan_hash, approver, timestamp)` y el agente en fase 2 verifica que `hash(plan)` coincide con uno aprobado.

Kata 08 — Memoria Jerárquica con `CLAUDE.md`

Concepto

`CLAUDE.md` es la memoria persistente del agente en un proyecto. Vive en tres niveles que se cargan en cascada:

- `~/.claude/CLAUDE.md` — preferencias del usuario.
- `<repo>/CLAUDE.md` — convenciones del equipo.
- `<repo>/<subpath>/CLAUDE.md` — reglas específicas de un módulo.

Y se compone modularmente con `@path/to/file.md`.

Por qué importa

Repetir convenciones en cada prompt cuesta tokens y diverge entre miembros del equipo. `CLAUDE.md` centraliza una sola fuente de verdad por nivel; el agente la carga automáticamente y la usa como contexto estable (caché-friendly, ver Kata 10).

Modelo mental

- Más específico gana. `repo/src/CLAUDE.md` sobreescribe `repo/CLAUDE.md`.
- Lo personal (estilo del usuario) NO va en el repo.
- Lo del equipo NO va en el home del usuario.
- `@imports` mantienen el archivo principal corto y permiten reutilización.

Ejemplo mínimo

```
# Project rules

## Style
@docs/style-guide.md

## Testing
@docs/testing.md

## Forbidden
- never run `pip install` without venv
```

Y en `~/.claude/CLAUDE.md`:

```
# My preferences
- prefer ruff over black
- terse commits
```

Ambos cargan; el del proyecto domina en conflictos sobre el proyecto.

Anti-patrón

- Meter preferencias personales en el `CLAUDE.md` del repo (contamina al equipo).
- Duplicar reglas en system prompt y CLAUDE.md (drift garantizado).
- Un `CLAUDE.md` monolítico de 2000 líneas (degrada caché y atención).

Argumento de certificación

- Sé describir el orden de precedencia entre los tres niveles.
- Sé justificar el uso de `@imports` para modularidad y caché.
- Sé enunciar qué pertenece a cada nivel y qué no.

Auto-evaluación

1. ¿Dónde guardo “este equipo usa pytest, no unittest”?

En el `CLAUDE.md` del proyecto (`<repo>/CLAUDE.md`). Convención del equipo, viaja con el repo.

2. ¿Dónde guardo “yo prefiero respuestas concisas”?

En `~/.claude/CLAUDE.md`. Es preferencia personal y no debe contaminar al equipo.

3. ¿Qué ocurre si el archivo del usuario y el del proyecto se contradicen?

Gana el del proyecto cuando se trata de algo del proyecto (lenguaje, lints, arquitectura). Gana el del usuario cuando es estilo conversacional. La resolución vive en `merge_claude_md`: el orden de los `layers=` en la llamada define la precedencia, y los layers más específicos van después para que su contenido aparezca al final del system prompt (zona de mayor atención, ver Kata 11).

Kata 09 — Reglas Condicionales por Ruta

Concepto

Reglas heurísticas (estilo, lints, convenciones específicas) se cargan **sólo cuando el agente edita archivos que coinciden con un patrón**. No siempre.

Por qué importa

Un `CLAUDE.md` que carga 2000 líneas para todos los archivos paga el costo en todas las sesiones, incluso cuando el agente sólo está editando un README. Cargar reglas Python sólo al tocar `*.py` libera contexto para el resto.

Modelo mental

- La regla declara su `glob` de activación: `src/**/*.py`, `*.tf`, etc.
- El agente carga la regla al entrar al archivo, la descarta al salir.
- Reglas grandes (heurísticas de lenguaje) → condicionales.
- Reglas universales (políticas de seguridad) → siempre cargadas.

Ejemplo mínimo

```
# CLAUDE.md (raíz)
@rules/security.md          # siempre

## When editing src/**/*.py
@rules/python-style.md
@rules/python-testing.md

## When editing infra/**/*.tf
@rules/terraform.md
```

`python-style.md` no se carga al editar `README.md` — no consume tokens.

Anti-patrón

Un único `CLAUDE.md` enorme con todas las heurísticas mezcladas. Cuesta tokens en cada turno, dispersa atención (ver Kata 11), y hace que cambiar una regla afecte sesiones que nada tienen que ver.

Argumento de certificación

- Sé clasificar qué reglas son condicionales y cuáles son universales.
- Sé estimar el ahorro de tokens al condicionar por ruta.
- Sé conectar este kata con Kata 8 (jerarquía) y Kata 10 (caché).

Auto-evaluación

1. ¿La regla “no commits sin tests” es condicional o universal?

Universal — aplica a todo cambio sin importar el lenguaje.

2. ¿Qué pasa si dos reglas condicionales se aplican al mismo archivo?

Ambas se concatenan al system prompt. Si se contradicen, es un bug del autor de las reglas — `assemble_system` debería detectar y advertir, o el equipo debe especializar el glob para no superponerse.

3. ¿Cómo verifico que la regla NO se cargó cuando no debía?

Comparando el system prompt resuelto entre dos targets: el caso B no contiene la sección “Python”. Una aserción

defensiva: `assert "type hints" not in assemble_system(["README.md"])`.

Kata 10 — Optimización Económica con Prefix Caching

Concepto

La API de Claude reusa el caché KV cuando el **prefijo** del prompt es idéntico turno a turno. Si organizamos el contexto como **estático primero, dinámico al final**, el primer ~90 % del prompt entra en caché y se factura ~10 % del costo.

Por qué importa

Insertar la fecha actual o el `user_id` al inicio del prompt invalida el caché en cada llamada. Mismo contenido, 10× más caro. El orden importa más que el volumen.

Modelo mental

- Estático = system prompt, CLAUDE.md, definiciones de tools, contexto pesado del repo.
- Dinámico = input del usuario, timestamps, estado efímero.
- Regla: estático arriba, dinámico abajo. El borde dinámico se aísla con tags XML `<reminder>`) para no ensuciar el prefijo.
- Métrica: `cache_creation_input_tokens` vs `cache_read_input_tokens`.

Ejemplo mínimo

```
messages = [
    {"role": "system", "content": SYSTEM_PROMPT_BIG_AND_STABLE},   # estático
    {"role": "user", "content": REPO_CONTEXT_BIG},                  # estático
    *prior_turns,                                                   # estático
    {"role": "user", "content": f"<reminder>now: {now}</reminder>\n{user_input}"},
]
```

Mismo system + mismo repo entre llamadas → cache hit.

Anti-patrón

```
content = f"Today is {today}. {SYSTEM_PROMPT}"   #    rompe caché
```

Cualquier valor que cambie por turno al inicio invalida todo lo que sigue.

Argumento de certificación

- Sé enunciar la regla “static-prefix-first, dynamic-suffix-last”.
- Sé interpretar las métricas `cache_creation` vs `cache_read`.
- Sé estimar el ahorro económico esperado (orden de magnitud ≈ 10×).

Auto-evaluación

1. ¿Dónde meto la fecha actual sin romper el caché?

En el sufijo del último mensaje del usuario, idealmente dentro de un tag (`<reminder>now=...</reminder>`). El prefijo (system + history estable) sigue siendo idéntico turno a turno.

2. Si el system prompt cambia un carácter, ¿qué pasa con el caché?

Se invalida — el prefijo deja de coincidir. Por eso los `@imports` (Kata 8) ayudan: cambias un archivo importado y solo se invalida si efectivamente el contenido cambia.

3. ¿Cómo demuestro empíricamente el ahorro?

Comparando `cache_read_input_tokens` entre la versión correcta y la rota. La diferencia es directa: en la celda §4 cache_read=0 en ambos turnos; en §3 cache_read >> 0 en el turno 2.

Kata 11 — Mitigación de Dilución Softmax (Edge Placement + Compactación)

Concepto

La curva de atención del transformer tiene forma de U: lo del inicio y el final del contexto se atiende fuerte; lo del medio se diluye ("lost in the middle"). Por eso las reglas duras se colocan en los **bordes** del prompt, y cuando el contexto pasa ~50 %, se **compacta** antes de que se pierdan.

Por qué importa

Un agente puede estar siguiendo una política perfectamente al turno 5 y violarla al turno 30 sin "olvidarla" — sólo dejó de atenderla porque quedó en el medio. La pérdida es silenciosa y no aparece en logs.

Modelo mental

- Bordes = atención alta. Centro = baja.
- Reglas críticas (seguridad, compliance) → al inicio Y se repiten al final como `<reminder>`.
- Detalles ricos (datos, código) → centro.
- Contexto > 50 % → compactar: resumen estructurado + scratchpad (Kata 18).
- "Compactar" no es "borrar"; es "reescribir en forma densa preservando invariantes".

Ejemplo mínimo

```
[SYSTEM]   reglas críticas duras                    ← borde
[CONTEXT]  archivos, datos, conversación pasada     ← centro
[USER]     pregunta actual
[REMINDER] mismas reglas críticas, reformuladas     ← borde
```

Política de compactación:

```
if usage_fraction(history) > 0.55:
    history = compact(history, preserve=["rules","decisions","escalations"])
```

Anti-patrón

Poner la regla más importante una sola vez, en medio de un system prompt gigante. El día que el contexto crece, la regla queda en el valle de la U y deja de aplicarse — sin error visible.

Argumento de certificación

- Sé describir la curva U y nombrar el efecto "lost in the middle".
- Sé enunciar la regla "bordes para reglas, centro para datos".
- Sé fijar un umbral concreto de compactación (50–60 %) y justificarlo.

Auto-evaluación

1. ¿Por qué repetir la misma regla al inicio y al final no es redundancia?

La curva U significa que ambos extremos tienen alta atención y el centro muy baja. Repetir la regla en los dos bordes garantiza que al menos una copia está en zona de alta atención sin importar dónde crece el contexto.

2. ¿Qué se compacta primero: turnos antiguos del usuario o tool_results intermedios?

Tool_results intermedios cuya conclusión ya quedó incorporada en turnos posteriores (ej. una búsqueda que ya

generó una decisión). Los turnos del usuario antiguos se compactan después, preservando los que contienen reglas o decisiones.

3. ¿Cómo pruebo que una regla “olvidada” se sigue aplicando tras compactar?

Inyectando el ataque (como `ATTACK` arriba) tras la compactación. Si la respuesta sigue rechazando, la regla sobrevivió. Si no, el preservador no la marcó como crítica — bug del filtro `preserve_keywords`.

Kata 12 — Prompt Chaining Multi-Pass

Concepto

Cuando una tarea no cabe cognitivamente (auditar 50 archivos, resumir 200 páginas), se descompone en **pases secuenciales**: pase local por unidad (archivo, página) y luego pase de integración que sólo ve los resúmenes.

Por qué importa

Pedirle al modelo "audita estos 50 archivos" en un solo prompt satura su atención: se pierde detalles, alucina entre archivos, y produce un resumen genérico. Encadenar prompts mantiene cada pase enfocado y barato.

Modelo mental

- Pase 1 (paralelo): por unidad, salida tipada y compacta.
- Pase 2 (integración): sólo ve los outputs del pase 1, no las unidades crudas.
- Cada pase tiene un schema de salida; el siguiente pase consume esos schemas.
- El modelo nunca ve la totalidad cruda.

Ejemplo mínimo

```
# Pase 1 (por archivo)
local = [analyze_file(f, schema=FileFindings) for f in files]

# Pase 2 (integración)
report = integrate(local, schema=AuditReport)
```

`local[i]` ya está condensado: el pase 2 cabe holgadamente.

Anti-patrón

Concatenar todos los archivos en un solo prompt y pedir el reporte final. Resultado: respuesta superficial, alucinaciones cruzadas entre archivos, costo máximo, calidad mínima.

Argumento de certificación

- Sé identificar tareas candidatas para chaining vs single-pass.
- Sé diseñar schemas de transición entre pases.
- Sé conectar este kata con Kata 4 (subagentes para paralelizar el pase 1) y Kata 11 (cada pase respeta el límite de contexto).

Auto-evaluación

1. ¿Cuándo NO conviene chaining (overhead > beneficio)?

Cuando los docs son muy pocos y muy cortos (<2-3 docs cabiendo holgadamente). El overhead de N llamadas no se justifica.

2. ¿Qué pasa si un pase 1 falla en una unidad? ¿se aborta, se omite, se reintenta?

Reintenta una vez (transitorio); si vuelve a fallar, lo registra con `error: true` en el agregado y deja que el pase 2 lo refleje en el reporte. Nunca se omite silenciosamente.

3. ¿Cómo evito que el pase 2 "rellene" lo que el pase 1 dejó vacío?

El system prompt del pase 2 es explícito: "no inventes información que no aparezca en los resúmenes". Y al ser tipados, el pase 2 puede ver `null` o `unclear` y respetarlos.

Kata 13 — Code Review Headless en CI/CD

Concepto

Claude Code corre en CI sin TTY (`claude -p ...`) y produce **JSON estructurado** con anotaciones por línea. El pipeline parsea el JSON con un schema y publica comentarios deterministas en el PR. No hay regex sobre prosa libre.

Por qué importa

Un reviewer humano se cansa, falla, y no escala a 100 PRs/día. Un reviewer LLM en CI encuentra issues consistentes (estilo, secrets, anti-patrones) en cada PR. Pero sólo si su salida es **estructurada** — si no, el pipeline tendrá que parsear prosa y romperá el primer día que el modelo cambie de redacción.

Modelo mental

- `claude -p "prompt" --output-format=json` → JSON contra schema declarado.
- Schema: lista de `Annotation { file, line, severity, rule_id, message }`.
- El runner del CI valida con el schema; si falla, falla el job, no se "ajusta".
- Humano sigue siendo el gate final de merge.

Ejemplo mínimo

```
# .github/workflows/review.yml
- run: |
    claude -p "$REVIEW_PROMPT" \
           --output-format=json \
           --schema annotations.schema.json \
           > out.json
- run: python scripts/post_annotations.py out.json
```

`post_annotations.py` valida con el schema y crea comentarios de PR. Cero parsing de texto.

Anti-patrón

Pedir review en prosa y hacer `grep "ERROR"` sobre el output. Falla el día que el modelo escriba "issue" o "warning" o cambie de idioma. Sin schema, el CI está apostando.

Argumento de certificación

- Sé describir el flag `--output-format=json` y la validación con schema.
- Sé conectar este kata con Kata 5 (extracción defensiva) y Kata 1 (control por señal, no por prosa).
- Sé justificar por qué el humano sigue siendo gate final.

Auto-evaluación

1. ¿Qué hace el CI si el JSON no valida contra el schema?

Falla el job, no "ajusta" la salida. Retorno code != 0 y el revisor humano interviene. Es preferible 1 PR sin review automatizado a 100 PRs con reviews silenciosamente vacíos.

2. ¿Cómo cacheo prompts caros en CI sin invalidar el caché por turno?

System prompt + reglas estables van con `cache_control: ephemeral` (Kata 10). El diff del PR va al final como `user` content. Mismo prefijo entre PRs distintos pero del mismo proyecto = cache_read alto.

3. ¿Qué reviews delego al modelo y cuáles dejo para humano?

Modelo: estilo, secrets, anti-patrones evidentes, lints semánticos. Humano: decisiones arquitectónicas, breaking changes, cambios en contratos públicos. La regla: si requiere conocimiento del negocio, humano.

Kata 14 — Few-Shot para Calibrar Bordes

Concepto

Cuando la tarea es subjetiva (tono, formato no estándar, juicio estético), una descripción zero-shot deja al modelo en su default genérico. **2–4 ejemplos input/output** desplazan su distribución hacia el formato deseado más rápido y más barato que un párrafo de instrucciones.

Por qué importa

Decir "responde en estilo casual chileno" no produce el resultado; mostrar 3 ejemplos de cómo se ve, sí. Few-shot es la forma más eficiente de comunicar "ground truth" para casos sin definición rígida.

Modelo mental

- Los ejemplos son del **mismo schema** que la salida esperada.
- Cubren los **bordes** del dominio, no el caso fácil.
- 2–4 suelen ser suficientes; >5 satura sin mejorar.
- Few-shot complementa (no reemplaza) al schema (Kata 5).

Ejemplo mínimo

```
prompt = """
Clasifica el ticket. Ejemplos:

ticket: "no me llega la factura desde hace 3 meses"
clase: "billing", urgencia: "high"

ticket: "tengo una sugerencia de mejora para la app"
clase: "feedback", urgencia: "low"

ticket: "no puedo entrar, me dice token expirado"
clase: "auth", urgencia: "high"

ahora clasifica:
ticket: "{user_text}"
"""
```

Anti-patrón

- Ejemplos triviales que no representan los bordes (todos casos fáciles).
- Llenar el prompt con 20 ejemplos "por si acaso": dispersa atención (Kata 11), rompe caché (Kata 10), no mejora.
- Mezclar formatos entre ejemplos.

Argumento de certificación

- Sé identificar cuándo few-shot supera a instrucciones en prosa.
- Sé diseñar ejemplos que cubran bordes, no centro.
- Sé combinar few-shot + schema (Kata 5) para tareas subjetivas con formato estricto.

Auto-evaluación

1. ¿Cuándo añadir un ejemplo más empeora el resultado?

Cuando satura la atención (más allá de ~4-5 para muchos casos) o introduce contradicciones con los anteriores. La

curva de mejora es cóncava: los primeros 2-3 valen oro; el quinto rara vez compensa.

2. ¿Por qué los ejemplos van al inicio (estático) y no al final?

Porque son contenido estable: van en el bloque cacheable (Kata 10) y en zona de alta atención (Kata 11). Si los pongo al final, los pierdo del caché y compiten con la pregunta del usuario.

3. Si los ejemplos contradicen el schema, ¿quién gana?

El schema. `tool_choice` con `input_schema` rechaza valores fuera del enum aunque el ejemplo los muestre. Por eso los ejemplos deben ser consistentes con el schema; un ejemplo que use una categoría no listada degrada los resultados.

Kata 15 — Evaluación Crítica y Auto-Corrección

Concepto

Cuando el modelo extrae números (totales, sumas, fechas calculadas), debe **cruzar lo que calculó vs lo que la fuente declara**. Si discrepan, no decide arbitrariamente: emite un flag de conflicto y enruta a revisión humana.

Por qué importa

Un total de factura calculado por el modelo puede coincidir con el declarado… o no. Sin verificación cruzada, el sistema confía silenciosamente en la alucinación más plausible. En facturación o contabilidad, eso es un incidente.

Modelo mental

- Dos fuentes de verdad: lo declarado en el documento y lo calculado por el agente. Deben coincidir.
- Si difieren más allá de un epsilon, marcar `mismatch=true` con ambos valores.
- Nunca "elegir el más razonable". Escalar (Kata 16).
- Aplica a: totales numéricos, sumas, conteos, fechas derivadas.

Ejemplo mínimo

```
{
  "stated_total": 1234.56,
  "computed_total": 1230.00,
  "mismatch": true,
  "delta": 4.56,
  "needs_human_review": true
}
```

Cliente:

```
if abs(stated - computed) > epsilon:
    flag_for_review(invoice, stated, computed)
```

Anti-patrón

Tomar `stated_total` directo o, peor, "corregirlo" silenciosamente al `computed_total` sin avisar. Puede ocultar fraude, errores de OCR, o alucinación del propio modelo.

Argumento de certificación

- Sé identificar campos numéricos sujetos a verificación cruzada.
- Sé definir el epsilon de tolerancia (cero para enteros, ε pequeño para monedas) y justificarlo.
- Sé conectar este kata con Kata 16 (escalada humana) y Kata 20 (provenance).

Auto-evaluación

1. ¿Qué pasa si el documento no declara total (sólo línea por línea)?

`stated_total = null` (campo nullable). `computed_total` se calcula. `mismatch = false` (no hay con qué comparar). `needs_human_review = false`. Estado limpio: el agente reporta sólo lo computado.

2. ¿Cómo distingo "error de OCR" de "fraude" en el flag?

A nivel del flag, no se distinguen — ambos disparan `needs_human_review`. La diferenciación es de proceso: la cola humana tiene heurísticas (delta pequeño y consistente con redondeos vs delta grande sin patrón).

3. ¿Qué prueba reintroduce el anti-patrón (auto-corrección silenciosa) y qué assert falla?

Modificar el system prompt a “si difieren, devuelve `stated_total` y no menciones la diferencia”. Una aserción defensiva en el cliente: si `stated_total` y `computed_total` están ambos presentes y difieren, `mismatch` debe ser `true`. Si el modelo pone `false` con valores discrepantes, falla el assert.

Kata 16 — Protocolo de Handoff a Humano

Concepto

Cuando el agente toca una política que no puede resolver (límite operativo excedido, decisión irreversible, conflicto de datos), invoca la herramienta `escalate_to_human`, **suspende generación de prosa** y emite un payload JSON estricto: `customer_id`, `issue_summary`, `actions_taken`, `escalation_reason`.

Por qué importa

Pasar al humano un transcript crudo de la conversación es desastre operacional: el operador tiene que leer todo, adivinar contexto, y decidir bajo presión. Un payload tipado le da un paquete autocontenido y accionable.

Modelo mental

- Detectar precondición → llamar tool `escalate_to_human`.
- El tool corta la generación de texto y obliga a salida tipada.
- El payload es **autocontenido**: el humano no debería tener que leer nada más.
- Es un end-state del bucle, no una pausa: el agente no continúa hasta que el humano decide.

Ejemplo mínimo

```
{
  "customer_id": "C-1287",
  "issue_summary": "Refund $1500 requested, exceeds tier-2 limit ($1000)",
  "actions_taken": ["validated_identity", "fetched_order_history"],
  "escalation_reason": "policy_limit_refund",
  "recommended_action": "human_approval_for_full_refund"
}
```

Anti-patrón

- Dejar que el modelo “negocie” en prosa con el cliente cuando ya superó la política.
- Pasar al humano `messages[]` crudo y que se las arregle.
- Resumen libre sin schema; el humano interpreta como puede.

Argumento de certificación

- Sé enumerar precondiciones de escalada típicas.
- Sé describir por qué la salida del handoff es tipada y autocontenida.
- Sé conectar este kata con Kata 2 (hook puede forzar `ask_human`) y Kata 15 (mismatch numérico → handoff).

Auto-evaluación

1. ¿Qué hago si el modelo intenta seguir generando prosa después del tool call?

El bucle del cliente, al detectar `tool_use=escalate_to_human`, despacha al humano y termina la sesión. No se inyecta ningún `tool_result` que reanude la conversación: el handoff es un end-state, no una pausa.

2. ¿Cómo aseguro que `actions_taken` refleja realmente lo ejecutado y no alucinación?

El cliente mantiene un log canónico de las acciones que se ejecutaron en el bucle (los `tool_use` aceptados con sus resultados). En el dispatcher, comparo `handoff["actions_taken"]` contra ese log y rechazo el handoff si menciona una acción que nunca corrió.

3. ¿Cuándo es legítimo no escalar y simplemente abortar?

Cuando el costo de resolver excede el valor del caso (peticiones spam, abuso). Aborto = respuesta fija “no procedemos” sin invocar al humano. Debe registrarse para que el operador, si revisa la cola de abortos por muestreo, detecte falsos positivos.

Kata 17 — Procesamiento Masivo con Message Batches API

Concepto

Para cargas que **no son interactivas** (auditorías, backfills, evaluaciones), la Message Batches API procesa miles de requests offline a ~50 % del costo. Cada request lleva un `custom_id` que correlaciona request↔response y aísla fallos.

Por qué importa

Pagar tarifa real-time por trabajo offline es desperdicio. Y procesar 10 000 prompts uno por uno con un `for` rompe rate limits, no maneja fallos parciales, y tarda horas. Batch es el patrón correcto.

Modelo mental

- Batch = colección de requests independientes con `custom_id` único.
- El batch puede acabar `ended` con éxitos parciales: cada request tiene su propio status.
- Se procesa en background, se polea, se descarga el resultado.
- Para fallos masivos, fragmentar el batch (sub-batches) y reintentar selectivamente.

Ejemplo mínimo

```
batch = client.messages.batches.create(requests=[
    {"custom_id": f"audit-{i}", "params": {...}}
    for i, _ in enumerate(items)
])
# poll
while batch.processing_status != "ended":
    sleep(30); batch = client.messages.batches.retrieve(batch.id)
# correlate
for r in client.messages.batches.results(batch.id):
    save(r.custom_id, r.result)
```

Anti-patrón

- Bucle síncrono `for item in 10_000: client.messages.create(...)` : tarifa full, rate limit, sin recuperación de fallos.
- Batch sin `custom_id` o con `custom_id` no único: pierdes correlación.
- Re-procesar todo el batch cuando sólo el 1 % falló.

Argumento de certificación

- Sé identificar cargas elegibles para Batch (offline, latency-tolerant).
- Sé describir el ciclo create → poll → results.
- Sé justificar la importancia del `custom_id` y la fragmentación selectiva.

Auto-evaluación

1. ¿Cuál es el ahorro económico esperado vs API real-time?

~50 % en input/output tokens (consultar pricing actual). Más el ahorro indirecto de no necesitar lógica de retry/backoff propia.

2. ¿Cómo recupero un batch interrumpido sin re-pagar el 99 % exitoso?

Filtrando los `result.type != "succeeded"` y armando un nuevo batch sólo con esos `custom_id`. El éxito previo no se re-procesa.

3. ¿Qué hago si dos requests del batch tienen el mismo `custom_id` por error?

La API lo rechaza al `create`. La defensa: validar localmente con `assert len({r["custom_id"] for r in reqs}) == len(reqs)` antes de enviar.

Kata 18 — Scratchpad Persistente

Concepto

Un archivo (`investigation-scratchpad.md`) externo a la conversación donde el agente vuelca **descubrimientos durables**: hipótesis confirmadas, decisiones, hallazgos de archivos. Sobrevive a `/compact` y a reinicios de sesión.

Por qué importa

Cuando el contexto se compacta (Kata 11), se pierde detalle. Si un descubrimiento crítico vivía sólo en el historial conversacional, desaparece. El scratchpad es memoria persistente curada por el propio agente.

Modelo mental

- Conversación = memoria volátil (puede compactarse o resetearse).
- Scratchpad = memoria persistente (archivo en disco).
- El agente escribe sólo conclusiones validadas, no monólogo.
- Estructurado en secciones: Hipótesis, Decisiones, Hallazgos, Pendientes.
- Al inicio de cada sesión nueva, el agente lee el scratchpad como contexto.

Ejemplo mínimo

```md
# Investigation Scratchpad

## Decisiones
- 2026-04-25: usar pydantic v2 (T-19 confirmó compatibilidad).

## Hallazgos
- `src/legacy/parser.py` tiene un bug de offset (línea 142). Replicado.

## Pendientes
- Revisar si `--strict` rompe tests integration-*.
```

Tool de escritura:

```python
def append_scratchpad(section: str, entry: str): ...
```

Anti-patrón

- “Mantener todo en la conversación, ya recordará”. Falla al primer `/compact`.
- Escribir el scratchpad en prosa libre y largo: pierde su función de memoria densa.
- Re-leer todo el scratchpad cada turno (rompe caché, ver Kata 10): leer al inicio y referenciar después.

Argumento de certificación

- Sé describir la diferencia entre memoria conversacional y memoria persistente.
- Sé enunciar qué se escribe en el scratchpad y qué no.
- Sé conectar este kata con Kata 11 (compactación) y Kata 19 (investigación adaptativa).

Auto-evaluación

1. ¿Qué pasa si el scratchpad y la conversación se contradicen?

Gana el scratchpad. La conversación es contexto reciente y volátil; el scratchpad es memoria curada. Si la contradicción es genuina, el agente debe actualizar el scratchpad con una nueva entrada que reemplaza la anterior.

2. ¿Cuándo el agente debe borrar** entradas del scratchpad?**

Cuando un hallazgo se invalida (ej. un bug reportado se confirma como falso positivo). En la práctica, agrego una entrada que invalida la anterior antes que borrar — preservar el rastro de la decisión.

3. ¿Cómo verifico que un hallazgo sobrevivió a `/compact`?

Inicio una sesión nueva (sin pasar `messages` previa), incluyo el scratchpad en el system prompt, y le pregunto al modelo por el hallazgo. Si lo recuerda, sobrevivió. La celda §3.1 hace exactamente eso.

Kata 19 — Investigación Adaptativa (Descomposición Dinámica)

Concepto

En un dominio desconocido, no se planifica al detalle de antemano: el agente **mapea topología** primero (búsqueda por nombre y por contenido), genera un plan priorizado, y **re-adapta** cuando un hallazgo invalida el plan vigente. Todo dentro de un presupuesto de exploración acotado.

Por qué importa

Un plan rígido en territorio desconocido garantiza desperdicio: el agente sigue explorando ramas muertas porque "estaba en el plan". Adaptar dinámicamente prioriza atención sobre lo que la realidad muestra, no lo que la hipótesis inicial asumía.

Modelo mental

- Fase 1: mapeo barato (glob de nombres, regex de imports/símbolos).
- Fase 2: priorización (¿qué módulos son centro? ¿cuáles huérfanos?).
- Fase 3: investigación profunda sólo sobre los priorizados.
- Si una hipótesis se rompe, **emitir nuevo plan** y volver a priorizar.
- Presupuesto: número máximo de archivos a leer / queries / minutos.

Ejemplo mínimo

```
topology = scan_repo(globs=["src/**/*.py"])
plan = prioritize(topology)               # heurística declarada
budget = Budget(files=50, queries=20)

while plan and budget.remaining():
    target = plan.pop()
    finding = deep_dive(target, budget)
    if finding.invalidates(plan):
        plan = re_plan(topology, finding)
```

Persistir en scratchpad (Kata 18) cada `finding` y `re_plan`.

Anti-patrón

- Hacer plan completo al minuto 0 y seguirlo aunque la realidad lo contradiga.
- "Read all files" sin presupuesto: se acaba el contexto antes de aprender algo.
- Re-planificar en cada turno por reflejo (no converge nunca).

Argumento de certificación

- Sé definir presupuesto de exploración y justificar el límite.
- Sé enunciar el criterio de re-planificación (qué disparara un re-plan, qué no).
- Sé conectar este kata con Kata 4 (subagentes para deep-dive) y Kata 18 (scratchpad como memoria del proceso).

Auto-evaluación

1. ¿Cómo decido si un hallazgo invalida el plan o sólo lo refina?

Refina si es un detalle dentro del scope actual ("la función está en `auth.py` no en `utils.py`"). Invalida si el scope

cambia ("el problema no es auth, es el middleware"). El umbral es práctico: si requiere mirar archivos no en el plan inicial, es invalidación → re-plan explícito.

2. ¿Qué hago cuando se acaba el presupuesto sin conclusión clara?

Devuelvo lo que tengo + flag `incomplete=true` + lista de pendientes para escalar. Nunca hago una conclusión inventada por presión de presupuesto.

3. ¿Cómo evito loops de re-planificación?

Cada re-plan consume del presupuesto. Si en N pasos no he reducido el espacio de exploración, abort. En código: contar re-plans y limitar a 2-3.

Kata 20 — Preservación de Provenance

Concepto

Cada afirmación factual extraída de fuentes mantiene un **mapeo tipado a su origen**: `claim`, `source_id`, `source_name`, `publication_date`. Los conflictos entre fuentes NO se resuelven en silencio: se marcan y se enrutan a humano.

Por qué importa

Tras agregar contenido de muchas fuentes vía subagentes (Kata 4), perder el hilo de "quién dijo qué" hace imposible auditar el resultado. Resúmenes en prosa libre **se ven correctos** y aluciinan sin que se note. Provenance es la única defensa.

Modelo mental

- No hay claim sin source. Es un invariante de schema.
- Si dos fuentes contradicen, se registran ambas bajo `conflict: true`, no se promedia ni se elige.
- La fecha de publicación importa: fuente más reciente no siempre gana, pero el humano necesita verla.
- El reporte final preserva la lista de claims tipados; la prosa, si existe, se genera**a partir** de esa lista, no la sustituye.

Ejemplo mínimo

```
{
  "claim": "ARR Q3 2025 = 12M USD",
  "sources": [
    {"id": "doc-A", "name": "Annual Report 2025", "date": "2025-12-01"},
    {"id": "doc-B", "name": "Investor Deck", "date": "2025-09-15"}
  ],
  "conflict": false
}
```

Conflicto:

```
{
  "claim": "Headcount end-2025",
  "sources": [
    {"id":"doc-A","name":"Report","date":"2025-12-01","value":"450"},
    {"id":"doc-C","name":"HR export","date":"2026-01-10","value":"462"}
  ],
  "conflict": true,
  "needs_human_review": true
}
```

Anti-patrón

- Resumen agregado en prosa, sin citas, sin source_id. Imposible auditar.
- "Resolver" el conflicto eligiendo la fuente que parece más oficial.
- Asumir que la fuente más nueva siempre gana.

Argumento de certificación

- Sé enunciar el invariante "no hay claim sin source".
- Sé describir la política de conflictos (registrar ambos, escalar).
- Sé conectar este kata con Kata 4 (agregación tras subagentes), Kata 15 (verificación numérica) y Kata 16

(escalada humana).

Auto-evaluación

1. ¿Qué hago si la fuente no tiene fecha?

`source_date` se marca null o `unknown`. El claim sigue siendo válido pero el humano que resuelve un conflicto verá la ausencia de fecha como debilidad de la fuente.

2. ¿Cuándo dos sources con el mismo `value` cuentan como "una sola confirmación" y cuándo como dos?

Si comparten origen primario (mismo dataset, mismo reporte oficial copiado en dos lugares), una. Si son extracciones independientes con cadenas de evidencia distintas, dos. La heurística práctica: registrarlos por separado y dejar que el humano lo lea.

3. ¿Qué prueba reintroduce el anti-patrón (prosa sin citas) y qué assert falla?

Reemplazar la salida del extractor por prosa libre. Una aserción defensiva: para cada claim final, `assert "source_id" in claim and claim["source_id"] in known_doc_ids`. Si el modelo emite un claim sin source_id (o con uno fabricado), el assert falla y el pipeline rechaza la respuesta.

Kata 21 — Calidad de Descripciones de Tools

Concepto

La descripción de un tool es **el único** mecanismo que el modelo usa para decidir cuál llamar. Una descripción mínima ("Analyzes content") deja al modelo adivinar entre tools similares. Una buena descripción incluye: input format, query examples, edge cases y la **frontera** explícita ("usa esto en lugar de X cuando…").

Por qué importa

El día que tienes `analyze_content` y `analyze_document` con descripciones genéricas, el modelo escoge mal en 20-30 % de los turnos. Y como el síntoma es "respuesta razonable pero del tool incorrecto", la falla no se ve en logs hasta que un downstream rompe.

Modelo mental

- Descripción = contrato de uso. Si dos tools tienen contratos solapados, son ambiguos por diseño.
- Renombrar es preferible a "explicar más" cuando los nombres son confusos (`analyze_content` → `extract_web_results`).
- Splitting beats overloading: una herramienta con 5 propósitos confunde; cinco con un propósito cada una son claras.
- El system prompt **interactúa** con la descripción: keywords del prompt pueden sesgar el routing.

Ejemplo mínimo

```
# Malo
{"name": "analyze_content", "description": "Analyzes content"}
{"name": "analyze_document", "description": "Analyzes documents"}

# Bueno
{
  "name": "extract_web_results",
  "description": (
    "Parses HTML pages from a search query into a list of "
    "{title, url, snippet} items. Use this when the input is a URL or "
    "raw HTML; for PDF/DOCX use parse_document instead."
  ),
}
```

Anti-patrón

Descripciones de una línea genéricas, nombres solapados, o un único tool "hacelo todo" que recibe un parámetro `mode`. Cualquiera de las tres fuerza al modelo a adivinar.

Argumento de certificación

- Sé enunciar la regla: descripciones son el árbitro de selección.
- Sé identificar tools ambiguos por contrato y proponer split o rename.
- Sé revisar el system prompt para detectar keywords que sesgan.

Auto-evaluación

1. Si dos tools quedan solapados, ¿prefiero renombrar o "explicar más"?

Renombrar primero. La causa raíz suele ser nombre confuso; descripciones extras no compensan un mal nombre. Si tras renombrar siguen ambiguos, splittear funcionalidad.

2. ¿Cómo mido empíricamente la tasa de tool misrouting?

Conjunto de queries con ground-truth de qué tool corresponde, ejecutar y comparar. La celda §3 hace exactamente eso a pequeña escala. En producción: log de `(query, tool_invoked, expected_tool)` con expected_tool inferido de la consecuencia downstream.

3. ¿Qué hago si el system prompt usa una keyword que el modelo asocia con el tool incorrecto?

Reescribir el prompt para no usar esa keyword o cambiar el nombre del tool. El examen ataca este caso con preguntas tipo "el system prompt dice 'analyze the content' y el tool `analyze_document` se llama por error" — la fix es evitar la colisión semántica.

Kata 22 — Configuración de MCP Servers

Concepto

Los servidores MCP se declaran en archivos JSON con dos scopes:

- **Project** (`.mcp.json`): compartido por el equipo vía git, define los servers que todo el repo necesita.
- **User** (`~/.claude.json`): personal del dev, no se versiona.

Las credenciales NUNCA se hardcodean: se inyectan con expansión de environment variables (`${GITHUB_TOKEN}`).

Por qué importa

Hardcodear un token en `.mcp.json` versionado equivale a publicarlo. Y poner reglas de equipo en `~/.claude.json` deja a los nuevos devs sin acceso. La elección de scope es la diferencia entre "funciona en todas las laptops del equipo" y "funciona sólo en la mía".

Modelo mental

- Project scope viaja con el repo, se descubre automáticamente al conectar.
- User scope es para experimentos personales que no afectan al equipo.
- Múltiples servers se descubren simultáneamente; el agente ve la unión de todos sus tools.
- MCP **resources** (catálogos de contenido) reducen llamadas exploratorias — cuando aplica, expón resources antes que tools.

Ejemplo mínimo

```
// .mcp.json (project, versionado)
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {"GITHUB_TOKEN": "${GITHUB_TOKEN}"}
    },
    "internal-docs": {
      "command": "node",
      "args": ["./scripts/mcp-docs.js"]
    }
  }
}
```

```
// ~/.claude.json (user, personal)
{
  "mcpServers": {
    "my-experimental-server": {"command": "python3", "args": ["./local-mcp.py"]}
  }
}
```

Anti-patrón

- Token hardcoded en `.mcp.json` versionado.
- Servers de equipo en `~/.claude.json` (no se replica al onboarding).
- Custom MCP server para algo que un built-in resuelve (Grep para buscar imports en el repo).

Argumento de certificación

- Sé distinguir cuándo project scope vs user scope.

- Sé enunciar el patrón de env-var expansion para credenciales.
- Sé argumentar cuándo usar MCP vs un built-in tool.

Auto-evaluación

1. ¿Qué pasa si dos servers exponen un tool con el mismo nombre?

Comportamiento undefined a nivel SDK; en práctica el último cargado gana o uno de los dos no se registra. La defensa: namespace en el nombre del tool (e.g., `github_create_issue` vs `jira_create_issue`).

2. ¿Cómo se manejan los secrets en CI cuando `.mcp.json` se ejecuta allí?

Los CI runners exponen secrets como env vars (GitHub Actions: `secrets.GITHUB_TOKEN`). El `${GITHUB_TOKEN}` en `.mcp.json` resuelve desde el runner, sin commit a git, sin exposure en logs.

3. ¿Cuándo prefiero un MCP resource sobre un MCP tool?

Cuando expongo un **catálogo de contenido** (issues, docs, schemas) que el agente necesita ver pero no actuar sobre — los resources se inyectan como contexto, evitando llamadas exploratorias innecesarias.

Kata 23 — Selección de Built-in Tools

Concepto

Claude Code/Agent SDK provee tools built-in para operaciones de filesystem y shell:

- **Grep**: buscar contenido (regex sobre archivos).
- **Glob**: buscar paths (patrones de nombre, `**/*.test.tsx`).
- **Read**: cargar archivo completo.
- **Edit**: modificación dirigida con anchor único.
- **Write**: sobrescribir; fallback cuando Edit no encuentra anchor.
- **Bash**: comandos shell.

Cada uno tiene un **uso primario** y un **failure mode** específico.

Por qué importa

Usar `Read` cuando quería `Grep` carga miles de tokens innecesarios. Usar `Edit` con un anchor no-único hace que la operación falle. Saber qué tool aplica a qué situación es mecánica básica que el examen prueba.

Modelo mental

Quiero...	Tool
Hallar dónde se llama `processRefund`	Grep
Listar todos los `*.test.tsx`	Glob
Cargar un archivo completo	Read
Cambiar una línea específica	Edit
Reescribir todo o si Edit falla	Write
Ejecutar `npm test`	Bash

Estrategia incremental: **Grep primero** para hallar entry points → **Read** para seguir imports → **Edit/Write** puntual. Nunca "leer todo el repo" upfront.

Ejemplo mínimo

```
# Buscar dónde se llama una función
grep(pattern="processRefund\\(", glob="**/*.py")

# Cargar un archivo de los hallados
read(path="src/billing/refund.py")

# Modificación dirigida (Edit)
edit(
  path="src/billing/refund.py",
  old_text="if amount > 1000:",
  new_text="if amount > MAX_REFUND:",
)
# Si old_text no es único o no existe → Edit falla
# Fallback: Read entero + Write completo
```

Anti-patrón

- Read sobre todos los archivos del repo "por si acaso".
- Edit con anchor genérico que matchea varias líneas.
- Bash para algo que Grep/Glob hace nativamente.

Argumento de certificación

- Sé escoger el tool correcto en una decisión rápida.
- Sé describir el failure mode de Edit y el fallback Read+Write.
- Sé defender una estrategia incremental (Grep → Read → Edit).

Auto-evaluación

1. ¿Qué hago cuando Edit falla por anchor no único?

Read del archivo, modificación local, Write completo. O añadir contexto al `old_text` (más líneas alrededor) hasta que sea único.

2. ¿Cómo investigo un repo desconocido sin leer todos los archivos?

Glob para mapear estructura → Grep para hallar entry points (función main, imports clave) → Read sólo de los archivos que Grep señaló como relevantes. Es la estrategia del Kata 19 (adaptive investigation).

3. ¿Bash es la respuesta correcta para "buscar todos los TODOs"?

No. Grep `pattern: "TODO"` es la opción nativa, más rápida y con output estructurado. Bash con `grep -r TODO .` funciona pero pierde la abstracción del SDK.

Kata 24 — Slash Commands Custom y Skills

Concepto

Claude Code permite extender la sesión con dos primitivas:

- **Slash commands** (`.claude/commands/X.md`): trigger explícito con `/X`.
- **Skills** (`.claude/skills/X/SKILL.md`): pieza on-demand con frontmatter que controla scope y tools.

Cada uno tiene scope project (compartido por git) y user (personal).

El frontmatter de SKILL.md acepta:

- `context: fork` — la skill corre en sub-agente aislado, no contamina la sesión principal.
- `allowed-tools: [...]` — lista whitelist; bloquea destructive si no se necesitan.
- `argument-hint: "..."` — texto que se le pide al dev cuando invoca sin args.

Por qué importa

Comandos en `~/.claude/commands/` no se replican al equipo cuando hacen `git pull`. Skills sin `context: fork` contaminan la sesión principal con output verbose. Sin `allowed-tools` una skill exploratoria puede borrar archivos por accidente.

Modelo mental

Necesidad	Mecanismo
`/review` para todo el equipo	`.claude/commands/review.md`
Macro personal	`~/.claude/commands/X.md`
Análisis verboso aislado	Skill con `context: fork`
Skill sólo lectura	Skill con `allowed-tools` sin Write
Convenciones siempre cargadas	CLAUDE.md, no skill
Workflow on-demand	Skill

Ejemplo mínimo

```
<!-- .claude/skills/codebase-analysis/SKILL.md -->
---
name: codebase-analysis
description: Mapea estructura de un módulo y devuelve resumen.
context: fork
allowed-tools: ["Read", "Grep", "Glob"]
argument-hint: "<dir-or-feature>"
---
# Skill body
1. Glob `**/{argument}/**/*.{py,ts,tsx}`
2. Grep imports cruzados
3. Devuelve resumen tipado
```

```
<!-- .claude/commands/review.md -->
Revisa el diff actual contra la guía de estilo del equipo.
Reporta findings en formato tipado.
```

Anti-patrón

- Skill exploratoria sin `context: fork` → contamina contexto principal con miles de tokens de discovery output.
- `~/.claude/commands/` para comandos del equipo → nadie más los recibe.

- Embeber convenciones siempre-aplicables en una skill on-demand → pertenecen a CLAUDE.md.

Argumento de certificación

- Sé escoger entre command vs skill por trigger y por scope.
- Sé enunciar para qué sirve cada frontmatter (`context`, `allowed-tools`, `argument-hint`).
- Sé conectar `context: fork` con la economía de contexto (Kata 11).

Auto-evaluación

1. Quiero un `/test-coverage` disponible cuando clonen el repo. ¿Dónde?

`.claude/commands/test-coverage.md`. Project-scoped, versionado. Disponible tras `git pull`.

2. Una skill exploratoria que devuelve 5000 tokens de discovery: ¿qué frontmatter aplica?

`context: fork` (corre aislada) y `allowed-tools: ["Read","Grep","Glob"]` (sin Write/Bash, exploración pura).

3. ¿Cuándo prefiero CLAUDE.md sobre una skill?

Cuando la convención debe estar **siempre cargada** (estilo, lenguaje, arquitectura). Skill cuando el workflow es **on-demand** (review, audit, análisis específico).

Kata 25 — Gestión de Sesiones (Resume y Fork)

Concepto

Tres patrones para preservar contexto entre llamadas:

- `--resume <name>` — continúa una sesión nombrada. Útil cuando el contexto previo sigue válido.
- `fork_session` — crea rama paralela desde una baseline para explorar enfoques divergentes (e.g., dos refactorizaciones distintas).
- **Sesión nueva con summary inyectado** — preferible cuando los tool results previos pueden estar stale (archivos modificados, datos cambiados).

Por qué importa

Resumir una sesión vieja con tool results stale lleva al modelo a referenciar archivos que ya no son lo que cree. Fresh + summary estructurado es **más confiable** que `--resume` cuando el mundo cambió.

Modelo mental

- **Resume**: contexto válido, conversación continúa lógicamente.
- **Fork**: dos caminos a explorar desde una misma baseline; cero interferencia entre ramas.
- **Summary nuevo**: tool results envejecieron; arrancar limpio inyectando los hallazgos clave en el system prompt.

Ejemplo mínimo

```
# Resume: misma investigación, sigue donde la dejé
claude --resume codebase-audit-2025-04

# Fork: exploro dos enfoques en paralelo
claude --fork codebase-audit-2025-04 --new-name approach-A
claude --fork codebase-audit-2025-04 --new-name approach-B

# Summary fresh: el repo cambió, no quiero stale tool results
SUMMARY=$(cat investigation-scratchpad.md)
claude -p "Continuamos la investigación. Hallazgos previos: $SUMMARY"
```

Anti-patrón

- `--resume` después de una refactorización masiva: el modelo recuerda archivos como eran antes.
- Crear forks que después se mezclan asumiendo que tienen contexto compatible.
- Inyectar el transcript completo viejo en una sesión nueva en lugar de un summary tipado.

Argumento de certificación

- Sé decidir resume vs fork vs new-with-summary en escenario.
- Sé identificar cuándo los tool results están stale.
- Sé conectar este kata con Kata 18 (scratchpad como source of summary).

Auto-evaluación

1. Mi compañero modificó 30 archivos. ¿Resume o fresh?

Fresh con summary. Los tool results de la sesión vieja (que leyó archivos versión A) ahora están stale.

2. Quiero comparar dos refactorizaciones desde la misma baseline.

`fork_session`. Cada rama explora un enfoque sin interferir con la otra.

3. ¿Cuándo `--resume` falla silenciosamente?

Cuando el modelo asume validez de tool results que ya no la tienen. El síntoma típico: el modelo cita un archivo o función que ya no existe.

Kata 26 — Validación y Retry con Error Feedback

Concepto

Cuando una extracción tipada falla validación (Pydantic / JSON Schema), no se la acepta como está ni se la repite ciega. Se hace **retry-with- error-feedback**: nueva llamada incluyendo el documento original, la extracción fallida, y el error específico de validación. El modelo auto-corrige.

Distinguir dos clases de fallo:

- **Recuperable** — error de formato/structure (fecha en `MM/DD/YYYY` cuando se pedía `YYYY-MM-DD`). Retry con feedback funciona.
- **No recuperable** — la información simplemente no está en la fuente. Retry sólo aluciinará.

Por qué importa

Reintentar sin feedback es ruido: el modelo no sabe qué cambiar. Aceptar la salida fallida silenciosamente rompe contratos downstream. Reintentar cuando el dato no existe garantiza fabricación.

Modelo mental

- Loop: extracción → validación → si error: extracción + error feedback → validación. Máximo 2-3 intentos.
- Cada intento añade **el error específico** del intento anterior, no un mensaje genérico "intenta de nuevo".
- Si tras N intentos no resuelve: marcar `needs_human_review` con la cadena de errores.
- Track `detected_pattern`: ¿cuál construct disparó el problema? Permite análisis sistemático cuando muchos extracts dismiss findings.

Ejemplo mínimo

```python
def extract_with_retry(client, doc, schema, max_retries=2):
    last_error = None
    extraction = None
    for attempt in range(max_retries + 1):
        feedback = ""
        if last_error:
            feedback = (
                f"\n\nIntento previo falló: {last_error}\n"
                f"Output previo: {extraction}\n"
                f"Corrige sólo lo que el error señala."
            )
        resp = client.messages.create(
            tools=[schema],
            tool_choice={"type": "tool", "name": schema["name"]},
            messages=[{"role": "user", "content": doc + feedback}],
        )
        extraction = next(b.input for b in resp.content if b.type == "tool_use")
        try:
            validate(extraction, schema)
            return {"extraction": extraction, "attempts": attempt + 1}
        except ValidationError as e:
            last_error = str(e)
    return {
        "extraction": extraction,
        "validation_error": last_error,
        "needs_human_review": True,
    }
```

Anti-patrón

- Retry con la misma prompt — el modelo no aprende qué cambiar.

- Aceptar la salida fallida.
- Reintentar indefinidamente cuando el dato no existe (alucinación garantizada).

Argumento de certificación

- Sé distinguir error recuperable de no recuperable.
- Sé describir el loop con feedback específico, no genérico.
- Sé conectar con Kata 15 (auto-corrección numérica) y Kata 16 (handoff).

Auto-evaluación

1. ¿Cuándo retry NO ayudará por más feedback que dé?

Cuando el dato simplemente no está en la fuente. El modelo no puede inventar y reintentar = invitación a alucinar.

2. ¿Por qué el feedback debe ser específico y no "intenta de nuevo"?

El modelo necesita saber **qué** cambiar. Sin el error específico, va a producir variaciones aleatorias.

3. ¿Qué pasa si tras 3 intentos sigue fallando?

Marcar `needs_human_review` con la cadena de errores. Detenerse — más intentos no resuelven y queman cuota.

Kata 27 — Multi-Pass Review e Independent Reviewer

Concepto

El modelo que **generó** código retiene el contexto de su propio razonamiento; eso lo hace mal revisor de su propio output. Una **instancia independiente** (sesión nueva, sin la cadena de generación) detecta más issues. Para PRs grandes, **multi-pass**: per-file analysis + cross-file integration pass.

Por qué importa

Pedirle al mismo agente que generó código que lo revise produce reviews superficiales o auto-justificantes. Y revisar 14 archivos en un solo pass dispersa atención: feedback inconsistente, bugs obvios omitidos.

Modelo mental

- **Self-review** ⊂ misma sesión. Limitada por contexto sesgado.
- **Independent reviewer** = sesión limpia que sólo ve el código resultante, no el razonamiento.
- **Multi-pass para PRs grandes**:
    - Pass A — per-file: profundidad local archivo por archivo.
    - Pass B — cross-file: integra los outputs de A para detectar interacciones.
- Ejecutar 3 pasadas independientes y "consensuar 2 de 3" suena bien pero **suprime issues raros legítimos**. No es solución.

Ejemplo mínimo

```python
def review_pr(client, files: dict[str, str]):
    # Pass A: cada archivo, sesión nueva por archivo
    per_file = []
    for path, content in files.items():
        per_file.append(review_file_independent(client, path, content))

    # Pass B: integra los hallazgos sin ver código crudo
    summary = json.dumps(per_file, ensure_ascii=False)
    return client.messages.create(
        system="Detecta interacciones cross-file y duplicados de findings.",
        messages=[{"role": "user", "content": summary}],
    )

def review_file_independent(client, path, content):
    # SESIÓN NUEVA: el reviewer no vio la generación
    return client.messages.create(
        system="Eres reviewer de seguridad y estilo. Devuelve findings tipados.",
        messages=[{"role": "user", "content": f"```python\n# {path}\n{content}```"}],
    )
```

Anti-patrón

- Generar código + pedirle al mismo agente que lo revise en el siguiente turno.
- Single-pass review sobre 14+ archivos.
- "Quórum 2-de-3 entre reviews" como filtro de calidad.

Argumento de certificación

- Sé enunciar por qué self-review es subóptimo.
- Sé separar per-file pass de cross-file integration pass.
- Sé argumentar contra el "quorum de N reviews" como solución.

Auto-evaluación

1. ¿Por qué el quórum de N reviews independientes hace daño?

Issues genuinos que aparecen sólo a veces (por non-determinism del modelo) son los**menos confiables de detectar a la primera** y los **más valiosos** cuando aparecen. Filtrar por consenso los descarta.

2. ¿Cómo aseguro que el reviewer no tiene la cadena de razonamiento del generador?

Sesión nueva. `messages=[]` empieza desde cero, sólo el código resultante en `user`. No incluir el system prompt original ni el assistant turn de generación.

3. ¿Cuándo el cross-file pass detecta algo que el per-file no?

Null prop entre módulos, race conditions en chains de funciones, contracts implícitos rotos (caller asume que callee garantiza X). Todo lo que requiere mirar dos archivos a la vez.

Kata 28 — Propagación de Errores Multi-Agente

Concepto

En sistemas hub-and-spoke, los errores deben propagarse al coordinador con **contexto estructurado**: `failure_type`, `attempted_query`, `partial_results`, `suggested_alternatives`. El coordinador decide si reintenta con otra query, sigue con resultados parciales, o aborta.

Cuatro reglas:

1. **Local recovery primero** — el subagente reintenta transients localmente; sólo propaga lo que no resuelve.
2. **Distinguir** access failure (timeout, permisos) de **valid empty result** (búsqueda exitosa, cero matches).
3. **Coverage gap annotation** en el output del synthesis.
4. **Nunca enmascarar** un error como success vacío.

Por qué importa

Devolver `{"results": []}` cuando un timeout impidió la búsqueda hace que el coordinador asuma "no había información" — y produzca un report confiado con un hueco silencioso. Generic `"search unavailable"` priva al coordinador del contexto para decidir alternativas.

Modelo mental

Subagente devuelve…	Coordinador hace…
`{success: true, results: [...]}`	Procesa normalmente
`{success: true, results: [], empty_valid: true}`	Marca topic como "sin matches"
`{failure_type: "timeout", attempted_query, partial_results, alternatives}`	Reintenta con alternative o sigue
`{failure_type: "permission", retryable: false}`	Aborta o escala

Ejemplo mínimo

```python
def web_search_subagent(query):
    try:
        results = http_search(query, timeout=10)
        if not results:
            return {"success": True, "results": [], "empty_valid": True, "query": query}
        return {"success": True, "results": results, "query": query}
    except TimeoutError:
        # Local recovery
        try:
            results = http_search(broaden(query), timeout=20)
            return {"success": True, "results": results, "query": broaden(query),
                    "note": "broadened after timeout"}
        except TimeoutError:
            return {
                "success": False,
                "failure_type": "timeout",
                "attempted_query": query,
                "partial_results": [],
                "suggested_alternatives": [broaden(query), simpler(query)],
            }
    except PermissionError as e:
        return {
            "success": False, "failure_type": "permission",
            "retryable": False, "explanation": str(e),
        }

def coordinator(topic):
    res = web_search_subagent(topic)
    if not res["success"] and res["failure_type"] == "timeout":
        # Reintenta con suggested_alternatives
        for alt in res["suggested_alternatives"]:
            r2 = web_search_subagent(alt)
            if r2["success"]: return r2
    return res
```

Anti-patrón

- Retornar `{"results": []}` en error de acceso (success vacío enmascarado).
- Generic `"search unavailable"` sin failure_type ni partial.
- Terminar el workflow al primer error de un subagente.
- Reintentar en el coordinador sin que el subagente intente local recovery.

Argumento de certificación

- Sé distinguir access failure de valid empty.
- Sé enunciar local recovery + structured propagation.
- Sé conectar con Kata 06 (estructura del error) y Kata 20 (coverage gap reporting).

Auto-evaluación

1. ¿Cómo distingue el coordinador "no había info" vs "no pudimos buscar"?

Por la combinación `success` + `empty_valid`. `success: true, empty_valid: true` = no había. `success: false, failure_type: "timeout"` = no pudimos.

2. ¿Quién intenta el primer retry: el subagente o el coordinador?

El subagente — local recovery primero. Sólo cuando agota localmente, propaga al coordinador con `suggested_alternatives` para que decida.

3. ¿Qué información mínima necesita el coordinador para decidir alternative recovery?

`failure_type`, `attempted_query`, `partial_results` (si los hay), `suggested_alternatives`. Sin alternatives el coordinador queda adivinando.

Kata 29 — Confidence Calibration y Stratified Sampling

Concepto

Para extracciones masivas (auditorías, batch processing), el modelo emite **field-level confidence scores** junto con cada valor. Esos scores se **calibran** contra un **labeled validation set**: la confianza self-reported sin calibrar está sesgada (suele ser sobreestimada).

Calibrado, los scores enrutan trabajo:

- **High confidence** → automatización + **stratified random sampling** para detectar patrones nuevos de error.
- **Low confidence** → revisión humana.

Y **siempre** medir accuracy por `document_type` y `field`, no agregada — el 97 % global puede ocultar 60 % en un segmento.

Por qué importa

Reportar "97 % accuracy global" y automatizar todo lo high-confidence suena seguro hasta que un tipo específico de doc falla en silencio. Stratified sampling sobre high-confidence es la red que detecta los nuevos modos de error que validation set viejo no captura.

Modelo mental

- Confidence raw del modelo ≠ probabilidad real de correctitud.
- Calibración: comparar score vs accuracy en validation set etiquetado por categorías de score.
- Stratified sampling: muestreo proporcional por document_type y por rango de score.
- Reportar accuracy desglosada; un agregado mente.

Ejemplo mínimo

```
EXTRACT_WITH_CONF = {
    "name": "extract_with_confidence",
    "input_schema": {
        "type": "object",
        "properties": {
            "value": {"type": ["string", "null"]},
            "field_confidence": {"type": "number", "minimum": 0, "maximum": 1},
        },
        "required": ["value", "field_confidence"],
    },
}

def calibrate(predictions, labeled_set):
    \"\"\"Devuelve mapping confidence_bucket -> empirical_accuracy.\"\"\"
    buckets = {0.5: [], 0.7: [], 0.9: []}
    for pred, truth in zip(predictions, labeled_set):
        for thresh in buckets:
            if pred["field_confidence"] >= thresh:
                buckets[thresh].append(pred["value"] == truth)
    return {t: sum(b) / len(b) if b else None for t, b in buckets.items()}

def stratified_sample(extractions, n_per_type=10):
    by_type = group_by(extractions, lambda e: e["doc_type"])
    sample = []
    for doc_type, items in by_type.items():
        high_conf = [i for i in items if i["confidence"] >= 0.9]
        sample += random.sample(high_conf, min(n_per_type, len(high_conf)))
    return sample
```

Anti-patrón

- Tomar `field_confidence` raw como probabilidad real sin calibrar.
- Reportar agregate accuracy como métrica única.
- Muestreo aleatorio simple (sin stratificar) — under-representa segmentos minoritarios donde el modelo puede fallar.

Argumento de certificación

- Sé enunciar la diferencia entre confianza raw y calibrada.
- Sé describir stratified sampling y por qué supera al simple random.
- Sé identificar cuándo un agregado de accuracy es engañoso.

Auto-evaluación

1. Mi modelo dice `confidence: 0.95` en una extracción. ¿Puedo automatizar?

No, no sin calibrar primero. Si el bucket [0.95, 1.0] tiene accuracy empírica de 99 % en validation, sí. Si tiene 80 %, automatizar = errores silenciosos.

2. ¿Cómo construyo un labeled validation set sin pagar mucho a humanos?

Stratified sampling intencional sobre tus extracciones existentes (buckets de confianza × document_type), n=50-200, etiquetar a mano. Es lo mínimo para una primera calibración.

3. ¿Por qué 97 % global puede ocultar problemas reales?

Si el 5 % de los docs son “tipo X” y el modelo acierta 50 % en ese tipo, el global se mantiene cerca de 97 % pero los errores se concentran en X. Reporting por document_type hace visible el problema.

Kata 30 — Criterios Explícitos para Reducir Falsos Positivos

Concepto

Instrucciones vagas ("be conservative", "only report high-confidence findings") fallan en producción: el modelo interpreta "conservative" distinto cada vez. Los criterios explícitos categóricos (con concretos ejemplos por nivel de severidad) producen clasificación consistente.

Y un punto operativo crucial: **un alto false positive rate en una sola categoría destruye la confianza del usuario en TODAS las categorías**. A veces conviene **deshabilitar** temporalmente la categoría problemática mientras se afina.

Por qué importa

Si el reviewer agente reporta "potential security issue" en código seguro 1 de cada 5 veces, los devs empiezan a ignorar todos los flags de seguridad — incluyendo los reales. La precisión es prerequisito de la utilidad.

Modelo mental

- "Confidence" como filtro NO funciona — el modelo está mal calibrado (Kata 29).
- Criterios deben ser **categóricos y concretos**: "report only when comments claim X but the actual code does Y", no "report inconsistent comments".
- Por categoría, definir **severidad con ejemplos de código** del nivel.
- Si una categoría arrastra falsos positivos → quitar/disable mientras se mejora; mantener las categorías de alta precisión activas.

Ejemplo mínimo

```
# Vago — falla a producir resultados consistentes
SYSTEM_VAGUE = (
  "Eres reviewer. Reporta findings de alta confianza. "
  "Sé conservador con los flags."
)

# Explícito — criterios categóricos con ejemplos
SYSTEM_EXPLICIT = """
Eres reviewer. Reporta findings sólo si cumplen UNO de estos criterios:

- security.hardcoded_secret: literal API key en el código.
  Ejemplo positivo: `OPENAI_KEY = "sk-abcdef..."`
  Ejemplo negativo: `OPENAI_KEY = os.environ["OPENAI_KEY"]`
- bug.null_deref: dereferencia un value sin chequeo cuando puede ser None.
  Ejemplo positivo: `user.name` con `user = db.find_one()` (puede None).
  Ejemplo negativo: `user.name` con previo `assert user is not None`.

NO reportes:
- Estilo (espacios, naming) — fuera de scope.
- Patterns "que podrían ser problemáticos" — sólo certezas.

Severidad: error (rompe runtime), warning (degrada en edge case), info (estilo). Reporta sólo error y warnir
"""
```

Anti-patrón

- "Be conservative" o "only report high-confidence" en el system prompt.
- Mezclar todas las categorías sin medir falsos positivos por categoría.

- Mantener una categoría que produce false positives “porque a veces acierta” — destruye confianza en las demás.

Argumento de certificación

- Sé reescribir un prompt vago en explícito categórico.
- Sé argumentar por qué medir falsos positivos por categoría (no agregada).
- Sé proponer disable temporal como estrategia de calibración.

Auto-evaluación

1. “Be conservative” — ¿por qué falla y qué lo reemplaza?

Falla porque el modelo interpreta “conservative” distinto cada vez. Reemplaza con criterios categóricos: “reporta SI X, NO reportes SI Y”, con ejemplos concretos de cada caso.

2. Mi reviewer reporta security findings con 35 % FP rate y bug findings con 5 %. ¿Qué hago primero?

Deshabilitar security temporalmente. La pérdida de 35 % FP en security está envenenando la utilidad del 95 % de bug findings correctos. Calibra security offline; mantén bug.

3. ¿Por qué severity-with-examples supera a “use your judgment”?

“Judgment” no es enseñable; ejemplos sí. Mostrar 2 ejemplos de “esto es error” + 2 de “esto no es error” calibra la distribución del modelo hacia el operacional del equipo, no su default genérico.

Banco de preguntas de práctica

Veinte preguntas multiple-choice en el formato exacto del examen. Cada una incluye:

- Setup (escenario en producción).
- 4 opciones A/B/C/D — los distractores son plausibles si entiendes el concepto a medias.
- **Respuesta correcta** y por qué cada distractor falla.

> Para que esto sea útil: tapa la respuesta antes de leerla. Si caes en un distractor, vuelve al kata correspondiente y al *Domain Deep Dive* del dominio.

Q1 (Domain 1 / Kata 01)

Tu agente de soporte llama a `process_refund` al final de un turno. La respuesta tiene `stop_reason="end_turn"` y el texto del modelo dice "voy a procesar el reembolso ahora mismo, listo." Tu bucle agéntico termina. ¿Qué afirmación describe correctamente lo que pasó?

A. El modelo procesó el reembolso porque el texto confirma la acción.
B. El modelo NO procesó el reembolso; `stop_reason="end_turn"` significa que terminó el turno sin invocar tool, y `process_refund` quedó sin ejecutar.
C. El bucle debe iterar una vez más para confirmar el reembolso.
D. `stop_reason="end_turn"` indica éxito de la ejecución del tool.

Respuesta: B

`stop_reason="tool_use"` es la única señal que indica que hay un tool para ejecutar. Con `end_turn`, el modelo cerró el turno; el texto prometiendo la acción es **prosa**, no señal. A confunde texto con ejecución (anti-patrón clásico). C suma iteraciones que no harán ejecutar lo que ya no se requirió. D es semánticamente falso — `end_turn` significa terminar, no éxito de tool.

Q2 (Domain 1 / Kata 02)

Un agente de support tiene `process_refund` y un system prompt: "NUNCA proceses reembolsos > $1000". En producción, el 3 % de reembolsos ejecutados están entre $1000 y $5000. ¿Cómo lo arreglas?

A. Reforzar el system prompt con MAYÚSCULAS y repetir la regla 3 veces.
B. Añadir few-shot examples mostrando que reembolsos > $1000 deben rechazarse.
C. Implementar un hook `PreToolUse` que valide `tool_input["amount"]` y deniegue si excede $1000.
D. Añadir una nota al final de cada user message recordando el límite.

Respuesta: C

Los reembolsos > $1000 son una regla de **negocio crítica**. Prompt-only (A, B, D) es probabilístico — fallará el día que un user message lo persuada o el modelo parafrasee la política. El hook es deterministic: el tool **no se ejecuta** si la condición no se cumple.

Q3 (Domain 1 / Kata 04)

Tu sistema multi-agente coordina 3 subagentes (web search, document analysis, synthesis). Notas que synthesis cita información que web search no le pasó. El log muestra que el coordinador pasa a synthesis **solo el output JSON tipado** de los otros dos. ¿Qué pasa?

A. Synthesis está alucinando porque no ve el contenido completo de los documentos; necesita herencia automática del historial del coordinador.
B. Synthesis tiene memoria entre invocaciones del coordinator que debe limpiarse.
C. Synthesis está alucinando; el coordinador debe pasar `relevant_excerpt` y `source_url` por cada finding, no solo el `claim`.
D. El subagente synthesis debe tener acceso directo al web search tool para verificar.

Respuesta: C

Hub-and-spoke estricto **prohíbe** herencia automática (descarta A). El subagente no tiene memoria entre invocaciones (descarta B). Darle acceso directo a herramientas que no son su especialidad (D) es exactamente el anti-patrón. La causa real: el coordinador está pasando `claim` pero no `relevant_excerpt` ni `source_url` — synthesis no tiene de dónde extraer la evidencia.

Q4 (Domain 2 / Kata 02 — Tool descriptions)

Tienes dos tools: `analyze_content` ("Analyzes content") y `analyze_document` ("Analyzes documents"). El 23 % de las invocaciones usa el tool incorrecto. ¿Primer paso?

A. Eliminar uno de los dos y usar uno solo para todo.
B. Reescribir las descripciones detallando input format, ejemplos de queries, edge cases y la frontera entre los dos.
C. Añadir un router que pre-clasifique inputs por keywords antes de exponerlos al modelo.
D. Bajar la temperatura del modelo a 0.

Respuesta: B

Las descripciones son el **único** mecanismo del LLM para escoger tool. Si son ambiguas, el modelo no tiene cómo distinguir. B es el cambio de mayor leverage. A puede tener sentido a largo plazo pero requiere rediseño; B es el "primer paso" pedido. C es over-engineering. D no ataca la causa raíz.

Q5 (Domain 2 / Kata 06 — MCP errors)

Tu MCP tool `search_db` falla intermitentemente. Hoy devuelve `{"error": "something went wrong"}`. El agente entra en bucle de retries que agotan budget. ¿Mejor cambio?

A. Devolver `{"isError": true, "errorCategory": "transient", "isRetryable": true, "retryAfterSeconds": 5}` y dejar que el cliente gestione retries con backoff.
B. Aumentar el max_tokens para que el modelo razone mejor sobre el error.
C. Añadir al system prompt: "si search_db falla, no reintentes más de 3 veces."
D. Capturar la excepción del lado del MCP y devolver `{"results": []}`.

Respuesta: A

Política de retry vive en el **cliente**, no en el modelo (descarta B, C). Devolver `[]` enmascara el fallo y el agente cree que la búsqueda fue exitosa pero vacía (descarta D — anti-patrón). A da metadata suficiente para que el cliente decida.

Q6 (Domain 2 / Kata 16 — tool_choice)

Tu agente de extracción debe llamar SIEMPRE al tool `extract_invoice` y nunca devolver texto. ¿Configuración correcta?

A. `tool_choice` no especificado.
B. `tool_choice = {"type": "auto"}`.
C. `tool_choice = {"type": "any"}`.
D. `tool_choice = {"type": "tool", "name": "extract_invoice"}`.

Respuesta: D

`auto` (default; A y B) deja al modelo decidir y puede devolver texto. `any` (C) garantiza que llame **algún** tool, pero si tienes 3 tools puede llamar otro. **Forced** con nombre garantiza que el tool específico se llame.

Q7 (Domain 3 / Kata 08 — CLAUDE.md hierarchy)

Un nuevo dev del equipo dice que Claude no respeta la convención de "siempre usar Pydantic en lugar de

dataclasses”. Otros devs sí la ven respetada. ¿Causa más probable?

A. El nuevo dev tiene una versión vieja de Claude Code.
B. La regla está en `~/.claude/CLAUDE.md` del dev que la creó, no en el `CLAUDE.md` del proyecto.
C. Los demás devs usan plan mode y el nuevo no.
D. El proyecto necesita un `.mcp.json` para servir la regla.

Respuesta: B

Configuración user-level (`~/.claude/CLAUDE.md`) es **local al usuario**. Convenciones de equipo van al CLAUDE.md del proyecto (versionado en git). El error clásico: el creador escribió la regla pensando que era del equipo pero la dejó en su home.

Q8 (Domain 3 / Kata 09 — Path-scoped rules)

Quieres que ciertas convenciones se apliquen a TODOS los archivos `*.test.tsx`, que están **dispersos** por todo el repo (al lado del código que testean). ¿Mecanismo correcto?

A. Un único `CLAUDE.md` raíz con la regla.
B. Un `CLAUDE.md` en cada subdirectorio que contenga tests.
C. Un archivo en `.claude/rules/` con frontmatter `paths: ["**/*.test.tsx"]`.
D. Un slash command `/lint-tests` que el dev invoque manualmente.

Respuesta: C

A carga la regla siempre, gastando tokens cuando no aplica. B es inviable: tests están dispersos, no en un árbol contenido. D es manual y olvidable. C es la herramienta exacta para este caso: glob- based conditional loading.

Q9 (Domain 3 / Kata 07 — Plan mode vs direct)

Tarea: refactorizar un monolito a microservicios; afecta 60+ archivos con decisiones sobre boundaries de servicios. ¿Modo correcto?

A. Plan mode — explorar el codebase, mapear dependencias, diseñar antes de tocar código.
B. Direct execution — empezar por mover funciones y dejar que el diseño emerja.
C. Direct execution con instrucciones detalladas upfront.
D. Direct execution; cambiar a plan si surgen problemas.

Respuesta: A

Plan mode existe exactamente para este tipo de tarea: cambios de gran escala con decisiones arquitectónicas. B asume que el diseño emergerá — costoso si las dependencias se descubren tarde. C asume que ya conoces la respuesta. D ignora que la complejidad ya está declarada en el requirement.

Q10 (Domain 3 / Custom slash command)

Quieres un `/review` disponible para todos los devs del equipo cuando clonen el repo. ¿Dónde lo creas?

A. `.claude/commands/review.md` (en el repo).
B. `~/.claude/commands/review.md` (home del dev).
C. En el `CLAUDE.md` del proyecto.
D. En un `.claude/config.json` con un array `commands`.

Respuesta: A

Project-scoped commands viven en `.claude/commands/`, versionados con el repo, disponibles tras `git pull`. B es para personal. C es para context/instructions, no comandos. D no existe.

Q11 (Domain 4 / Kata 05 — Schema design)

Extraes facturas. Algunas no tienen `due_date`. El agente está inventando fechas plausibles cuando no hay. ¿Cambio?

A. Añadir un few-shot example mostrando una factura sin due_date.
B. Marcar `due_date` como nullable union (`{"type": ["string","null"]}`) y declararlo opcional, no required.
C. Bajar la temperatura.
D. Aumentar el max_tokens.

Respuesta: B

`required` fuerza al modelo a poner algo → alucina. Nullable union + opcional le da una vía honesta para representar ausencia. A puede ayudar pero no es la causa raíz: el schema mismo lo está empujando a inventar.

Q12 (Domain 4 / Kata 14 — Few-shot)

Clasificas tickets en 5 categorías. Algunos tickets ambiguos ("podría ser billing o auth") suelen mal clasificarse. ¿Mejor intervención?

A. Añadir 2-3 few-shot examples de tickets ambiguos mostrando el razonamiento de por qué se eligió cierta categoría.
B. Añadir 20 examples cubriendo todas las categorías.
C. Bajar la temperatura.
D. Cambiar a un modelo más grande.

Respuesta: A

Few-shot bien construido (B) cubre los **bordes**, no el centro. 2-4 ejemplos de bordes con razonamiento explícito enseñan generalización. 20 ejemplos saturan atención (Kata 11).

Q13 (Domain 4 / Kata 15 — Validation/retry)

Una extracción Pydantic-validada falla porque el modelo devolvió fechas en formato `MM/DD/YYYY` cuando el schema pide `YYYY-MM-DD`. ¿Estrategia?

A. Reintentar con la misma prompt; el modelo "verá" el error.
B. Reintentar incluyendo el doc original, la extracción fallida, y el specific validation error como feedback.
C. Aceptar `MM/DD/YYYY` también; no es grave.
D. Devolver error y enrutar a humano.

Respuesta: B

Retry-with-error-feedback es el patrón canónico para errors de **formato/structure** (resolvable por el modelo si se le dice qué falló). A no funciona si el modelo no sabe qué cambiar. C debilita el contrato downstream. D es overreach para un fix automático disponible.

Q14 (Domain 4 / Kata 17 — Batch API)

Tu manager propone migrar a Batch API tanto los pre-merge checks (blocking, devs esperando) como los reportes de tech debt overnight. ¿Cómo evalúas?

A. Migrar ambos; ahorra 50 % en costo.
B. Migrar solo los reportes overnight; mantener pre-merge en API real-time.
C. Mantener ambos en real-time para evitar reordenamiento de resultados.
D. Migrar ambos con timeout fallback a real-time.

Respuesta: B

Batch tiene ventana hasta 24h sin SLA. Bloquear a un dev en pre-merge mientras Batch procesa es inaceptable. Reportes overnight encajan perfectamente. A ignora la latencia. C deja dinero en la mesa cuando hay un workflow ideal para batch. D agrega complejidad innecesaria.

Q15 (Domain 4 / Kata 13 — Multi-pass review)

Un PR modifica 14 archivos. Tu review single-pass produce feedback inconsistente: detallado en algunos, superficial en otros, contradictorio entre archivos. ¿Restructuración?

A. Pasar por archivo individualmente, luego un pass de integración que mire data flow cross-file.
B. Pedirle al dev que parta el PR en chunks de 3-4 archivos.
C. Usar un modelo con context window más grande.
D. Correr 3 reviews independientes y reportar solo issues que aparezcan en al menos 2.

Respuesta: A

Causa raíz: attention dilution sobre 14 archivos. A ataca la causa con **multi-pass**: profundidad por archivo + integración cross-file en pass dedicado. B desplaza el problema al dev. C confunde tamaño de contexto con calidad de atención. D suprime issues que sólo se detectan intermitentemente — pierde recall.

Q16 (Domain 5 / Kata 11 — Context management)

Una sesión de soporte de 30 turnos: el agente empieza a referenciar "typical patterns" en lugar de los detalles específicos del caso. ¿Causa más probable?

A. El modelo está confundido; subir temperatura.
B. Context degradation: los detalles transaccionales (order_id, amount, status) se diluyeron en summary; falta un bloque de "case facts" persistente en cada prompt.
C. Necesitas cambiar a un modelo con context window más grande.
D. Hay que reiniciar la sesión.

Respuesta: B

El patrón "typical" en lugar de "specific" es la huella de summarización progresiva. Solución: extraer datos transaccionales a un **case facts block** que se inyecta en cada prompt fuera del summary. C no resuelve la calidad de atención. D pierde contexto.

Q17 (Domain 5 / Kata 16 — Escalación)

Un cliente dice "estoy frustrado, quiero hablar con un humano". El issue es estándar (cambio de password, hay procedimiento claro). ¿Acción correcta?

A. Resolver el issue autónomamente, ignorando la petición.
B. Reconocer la frustración pero ofrecer resolver; escalar sólo si el cliente reitera la petición.
C. Honrar la petición de humano inmediatamente sin investigar.
D. Pedirle al cliente que confirme con un email.

Respuesta: B

Cuando el issue es **straightforward** y el cliente expresa frustración pero el agent puede resolverlo, ofrecer resolución acknowledging la frustración es óptimo — escalar sólo si reitera la petición. C es válido si el cliente **explícitamente y firmemente** demanda humano (no aplica aquí). A ignora una preference del cliente. D introduce fricción innecesaria.

Q18 (Domain 5 / Kata 06 — Error propagation multi-agente)

Un subagente de web search timeoutea. ¿Diseño de propagación correcto?

A. Subagent retorna `{"results": []}` marcado como successful.
B. Subagent propaga la excepción al top-level, terminando el workflow.
C. Subagent retorna structured error `{failure_type, attempted_query, partial_results, suggested_alternatives}` al coordinador.
D. Subagent reintenta indefinidamente hasta éxito.

Respuesta: C

Structured error context permite recovery decisions inteligentes en el coordinador. A enmascara el fallo (anti-patrón). B termina todo cuando había recovery posible. D bloquea el workflow.

Q19 (Domain 5 / Kata 20 — Provenance)

Synthesis recibe findings de 2 subagentes con cifras de revenue diferentes (12M vs 12.4M, ambas de fuentes credibles). ¿Acción?

A. Promediar a 12.2M y reportar.
B. Reportar la cifra del documento más reciente.
C. Estructurar el output preservando ambos valores con su source attribution; marcar conflict; no resolver arbitrariamente.
D. Pedir a un subagente que decida cuál es correcta.

Respuesta: C

Conflictos se preservan, no se resuelven. Promediar (A) introduce un número fabricado. "Más reciente gana" (B) no es siempre cierto y oculta el conflicto. D es delegación sin información — el subagente no tiene mejor base para decidir.

Q20 (Domain 5 / Kata 18 — Scratchpad)

Una investigación de codebase de varias horas alcanza el límite de contexto. `/compact` viene; vas a perder hallazgos detallados. ¿Mejor defensa preventiva?

A. Subir el max_tokens.
B. Mantener un scratchpad file con findings clave (qué funciones importan, decisiones tomadas) que el agente lee al reanudar.
C. Reiniciar la sesión cada hora.
D. Imprimir todo el output a un log al terminar.

Respuesta: B

Scratchpad estructurado sobrevive `/compact` y reanudaciones — es la contramedida específica para context degradation en investigaciones largas. A no resuelve nada. C pierde contexto. D no recupera nada en la sesión actual.

Topics bonus — examen-relevantes pero no profundos en katas

Estos aparecen en el examen y los katas los tocan tangencialmente. Vale la pena memorizarlos:

Session resumption y forking (Agent SDK / Claude Code CLI)

Comando	Para qué
`--resume <session-name>`	Continúa una sesión nombrada (named resume)
`fork_session`	Crea una rama paralela desde una baseline
Sesión nueva + summary inyectado	Mejor que resume cuando los tool results stale

Decisión clave: si el contexto previo sigue *válido*, resume. Si los tool results pueden estar stale (archivos modificados, datos cambiados), **comienza nueva** con summary inyectado.

Skills frontmatter (Claude Code)

```
---
name: explore-architecture
context: fork                    # corre en sub-agente aislado
allowed-tools: ["Read","Grep","Glob"]
argument-hint: "<dir-or-feature>"
---
```

- `context: fork`: la skill corre en sesión separada; su output verbose no contamina la principal.
- `allowed-tools`: restricción de tools durante la skill. Útil para prevenir destructive actions.
- `argument-hint`: prompt al dev cuando invoca la skill sin args.

Comandos de Claude Code

Comando	Uso
`/memory`	Ver qué CLAUDE.md y rules están cargados
`/compact`	Comprimir context manualmente
`/resume`	Reanudar sesión nombrada

Tools built-in vs MCP — cuándo cada uno

- **Built-in**: Read, Write, Edit, Bash, Grep, Glob — cubren operaciones de filesystem y shell. Fast, no requieren config.
- **MCP**: integraciones externas (GitHub, Jira, DBs custom). Configurar via `.mcp.json` con env-var expansion para tokens.

> **Anti-patrón**: usar MCP para algo que un built-in resuelve. Si un dev pregunta "necesito buscar imports en el repo", la respuesta es Grep, no un MCP server custom.

Confidence calibration y stratified sampling

Para extracciones automatizadas en producción:

- **Field-level confidence** (output del modelo) calibrado contra un **labeled validation set** — la confianza self-reported sin calibrar está sesgada.
- **Stratified random sampling** sobre extracciones high-confidence: permite detectar nuevos patrones de error que la validación normal no captura.

- Reportar accuracy por **document type** y **field segment**, no agregada — el 97 % global puede ocultar 60 % en un tipo de doc.

Coverage gap reporting (Multi-agent)

Synthesis output debe estructurarse con secciones que distinguen:

- **Well-supported findings** (varias fuentes confirman).
- **Contested findings** (fuentes disagree; preservar ambas).
- **Coverage gaps** (topic donde las fuentes no devolvieron data).

> Si un subagente no encuentra info, el output del synthesis debe reflejar el gap, no rellenar.

Glosario

`@import`. Sintaxis en CLAUDE.md para incluir otros markdown files modularmente. Permite reutilizar standards entre proyectos.

`AgentDefinition`. Configuración del Agent SDK por subagente: descripción, system prompt, tool restrictions.

`allowedTools`. Configuración del coordinador que restringe qué tools puede usar. Para spawnear subagentes vía `Task`, debe incluir `"Task"`.

Agent SDK (Claude Agent SDK). Librería para construir agentes custom. Provee bucles agénticos, hooks (`PostToolUse`, tool call interception), spawning de subagentes.

Anti-patrón. Implementación que parece funcionar y falla en producción. Cada kata documenta uno; el examen los detecta.

Bootstrap (`katas._shared.bootstrap`). Helper del workshop que pide la API key y construye un cliente con presupuesto.

Budget guard. Wrapper sobre `client.messages.create` que aborta después de N llamadas, previniendo loops costosos.

Cache control (`cache_control: ephemeral`). Marca un bloque como cacheable a nivel SDK. Implementa prefix caching.

Case facts block. Sub-bloque persistente del prompt con datos transaccionales (order_id, amount, status) que sobrevive a la summarización progresiva.

`CLAUDE.md`. Configuración persistente. Tres niveles: user (`~/.claude/CLAUDE.md`), project (`.claude/CLAUDE.md` o `CLAUDE.md`), directory (subdirs).

Claude Code. CLI/IDE para desarrollo asistido. Diferente del Anthropic API SDK.

Coordinator-subagent. Patrón hub-and-spoke. El coordinador delega, no comparte historial automáticamente.

`context: fork`. Frontmatter de SKILL.md que ejecuta la skill en sub-agente aislado.

`custom_id`. Identificador único por request en Message Batches API. Permite correlación 1:1 input↔output.

Determinismo. Mismas señales → mismas decisiones. Sólo metadata estructurada (`stop_reason`, hook verdict) lo garantiza.

Edge placement. Reglas críticas al inicio Y al final del contexto para sobrevivir lost-in-the-middle.

Escape enum. Enum value (`other`, `unclear`) acompañado de un detail string. Permite al modelo admitir ambigüedad.

`errorCategory`. Campo MCP estructurado: `transient | validation | business | permission`. Cada categoría tiene política de retry distinta.

`fork_session`. Crea rama paralela desde una baseline para explorar enfoques divergentes.

Hub-and-spoke. Topología: coordinador central, N subagentes aislados, cero aristas laterales.

`isError` / `isRetryable`. Flags MCP en el resultado de un tool. Cliente decide retry/escalate desde estos flags, no desde prosa.

`isolation_fork` / `--resume`. Mecanismos de session management.

Lost in the middle. Atención U-shape: bordes alta, centro baja.

MCP (Model Context Protocol). Protocolo estándar para tools y resources externos. Servidores se configuran en `.mcp.json`.

Multi-pass review. Revisar PRs grandes en pasadas: per-file analysis + cross-file integration pass.

Plan mode. Modo read-only para tareas con decisiones arquitectónicas. Cliente entrega tools distintas en cada fase.

Prefix caching. Reutilización del KV cache cuando el prefijo del prompt coincide turno a turno. Estático arriba, dinámico abajo.

`PreToolUse` / `PostToolUse` hook. Funciones que corren antes (gate) o después (normalización) de la herramienta.

Provenance. Mapeo `claim → source` preservado. Conflictos se mantienen anotados, no se resuelven.

Scratchpad. Archivo markdown estructurado con findings durables. Sobrevive `/compact` y resume.

Skill (`.claude/skills/X/SKILL.md`). Pieza on-demand con frontmatter (`context`, `allowed-tools`, `argument-hint`).

`stop_reason`. Metadata estructurada del Message: único árbitro del control de flujo.

Stratified random sampling. Muestreo proporcional sobre extracciones high-confidence para detectar nuevos errores.

Subagent. Llamada nueva e independiente con prompt mínimo y salida tipada. Cero memoria heredada.

`Task` tool. Mecanismo del Agent SDK para spawnear subagentes desde el coordinador.

`tool_choice`. Configuración: `auto` (default), `any` (fuerza algún tool), forced (`{"type":"tool","name":"X"}`).

`tool_use` block. Bloque del response cuando `stop_reason="tool_use"`. Contiene `name`, `input`, `id`.

Cheat sheet (una página)

#	Kata	Regla central
01	Bucle agéntico	Control por `stop_reason`, jamás por texto del modelo
02	PreToolUse	Política en código, prompt sólo sugiere
03	PostToolUse	El modelo nunca ve datos crudos heterogéneos
04	Subagent isolation	Hub-and-spoke; subagente recibe prompt mínimo, devuelve JSON tipado
05	Defensive extraction	Schema con `tool_choice` forzado, enum con escape, nullable union
06	MCP errors	`isError` + `errorCategory` + `isRetryable` tipados, retry en cliente
07	Plan Mode	Tools de lectura → plan firmado → tools de escritura
08	CLAUDE.md memory	Cascada user → project → module; `@imports` para modularidad
09	Path rules	Reglas heurísticas se cargan sólo cuando el target hace match
10	Prefix caching	Estático arriba (cacheable), dinámico abajo
11	Softmax mitigation	Reglas duras en bordes; compactar al 50–60 % de capacidad
12	Prompt chaining	Pase 1 tipado por unidad → pase 2 integra sin ver crudos
13	Headless CI review	`claude -p` con schema; `jsonschema.validate` antes de publicar
14	Few-shot	2-4 ejemplos de bordes, no del centro fácil
15	Self-correction	`mismatch` y `needs_human_review` son required, no se autocorrige
16	Human handoff	Tool con schema estricto y enum de motivo; suspende generación de prosa
17	Batch processing	`custom_id` único, polling/webhook, recuperación selectiva
18	Scratchpad	Memoria persistente fuera de la conversación; sobrevive a `/compact`
19	Adaptive investigation	Topology → prioritize → deep-dive con presupuesto y re-plan
20	Data provenance	Cada claim con `source_id`; conflictos preservados, no resueltos
21	Tool descriptions	Descripción = árbitro de selección; explícita y diferenciadora
22	MCP server config	Project (`.mcp.json`) vs user (`~/.claude.json`); env-var expansion
23	Built-in tools	Grep contenido, Glob paths; Edit fail → Read+Write; incremental
24	Slash commands & skills	`.claude/commands/` proyecto; SKILL.md frontmatter `context: fork`
25	Session management	Resume si válido; fork para paralelo; fresh+summary si stale
26	Validation-retry	Loop con feedback específico; recuperable vs ausente
27	Multi-pass review	Independent reviewer (sesión nueva); per-file + cross-file
28	Multi-agent errors	Distinguir failure vs empty_valid; local recovery + structured prop.
29	Confidence calibration	Calibrar conf raw vs labeled set; stratified sampling
30	Explicit criteria	Categorías con criterios “report SI / NO SI” + ejemplos; FP por cat.

Diez reglas para repetir antes del examen

1. **Control por señal**, no por prosa.
2. **Hooks, no prompts** para reglas críticas (financial, identity).
3. **Schemas fallan cerrados**; nullable union antes que required-fabricado.
4. **Subagentes con prompt mínimo y output tipado**; cero historial heredado.
5. **Estático arriba, dinámico abajo**; reglas en bordes.
6. **Errores tipados** (`isError`, `errorCategory`, `isRetryable`), retry en cliente.
7. **Plan mode** ante decisiones arquitectónicas; **direct** ante scope claro.
8. **Few-shot 2-4 de bordes**, schema-consistent.
9. **Batch para no-blocking**; synchronous para blocking.
10. **Provenance preservada**, conflictos anotados, no resueltos.

Diez anti-patrones para reconocer al instante

Síntoma	Anti-patrón
`if "done" in text`	Prose-based termination
Política sólo en system prompt	Prompt-only enforcement
Subagente recibe history del coordinador	Telepatía compartida
`required: true` para todos los campos	Forced fabrication
Tool error como string `"failed"`	Untyped error
Reembolsos > X aprobados con prompt warning	Soft control on hard rule
Timestamp al inicio del system prompt	Cache-breaking
Regla crítica en mitad de un prompt largo	Lost-in-the-middle
18 tools al mismo agente	Tool overload
Resumen agregado sin source_id	Provenance loss

Out-of-scope (NO entra en el examen)

No pierdas tiempo estudiando esto:

- Fine-tuning o entrenamiento de modelos custom.
- Authentication, billing, o account management de la Claude API.
- Implementación detallada en lenguajes/frameworks específicos (más allá de tools y schemas).
- Deploying o hosting de MCP servers (infra, networking, contenedores).
- Arquitectura interna de Claude, training process, model weights.
- Constitutional AI, RLHF, safety training.
- Embedding models o vector DB internals.
- Computer use (browser automation, desktop interaction).
- Vision/image analysis.
- Streaming API o server-sent events.
- Rate limiting, cuotas o cálculos de pricing.
- OAuth, rotación de API keys, protocolos auth.
- Configuraciones específicas de cloud (AWS/GCP/Azure).
- Performance benchmarking entre modelos.
- Detalles de implementación de prompt caching más allá de “existe”.
- Algoritmos de tokenización.

> **Lo que sí cubre el examen** (resumen): ver “Examen de un vistazo” al inicio. Si una pregunta empieza con “you are deploying an MCP server on Kubernetes…”, revisa el wording — probablemente es una distracción que apunta a la decisión arquitectónica (qué scope usar), no a la mecánica de despliegue.